INTEGRAÇÃO E ENTREGA CONTÍNUA

Relatório de evidências Etapas 1 a 5

Projeto Cadastro de Pessoas em JavaScript

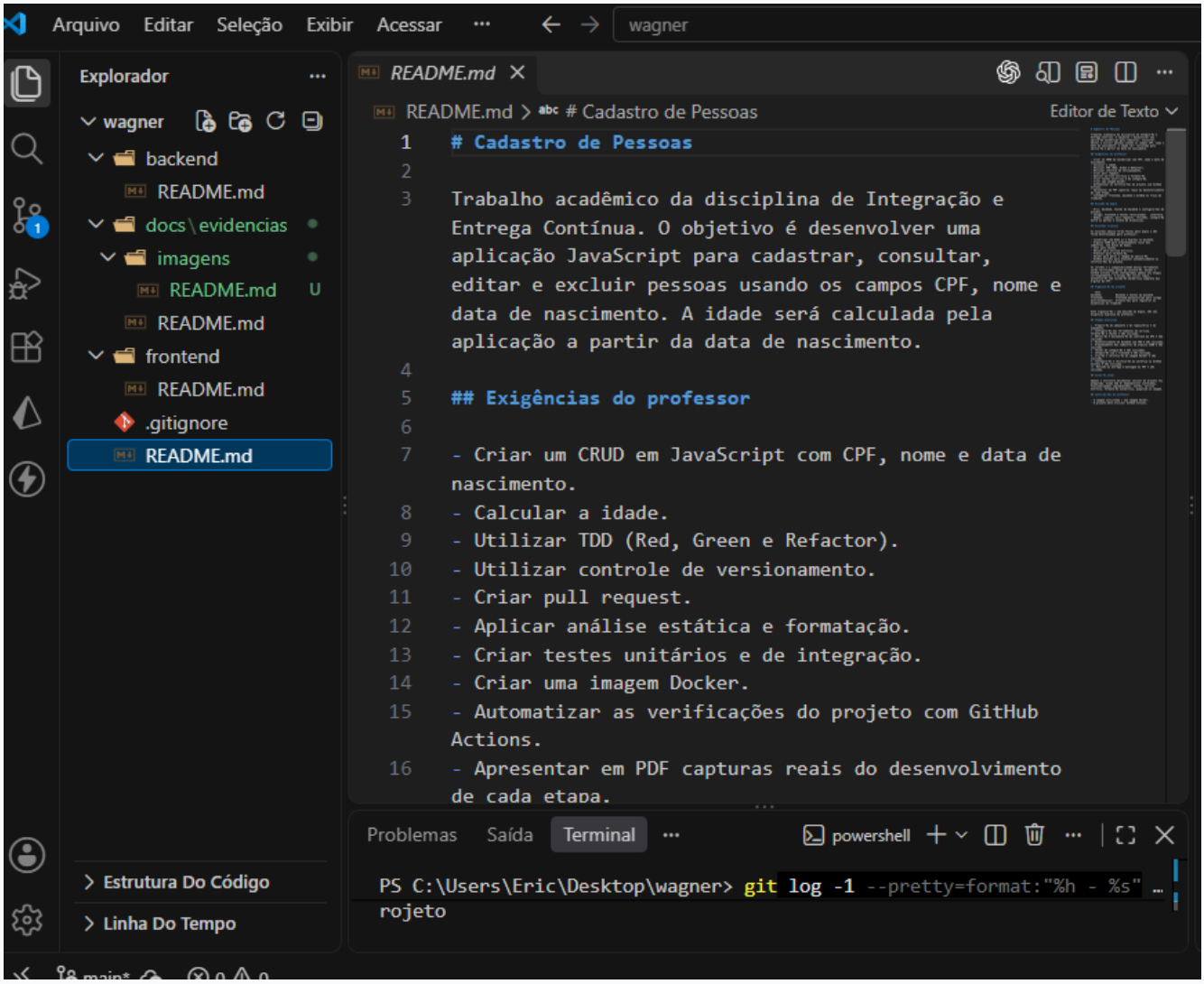
> Preparação do projeto, ferramentas, colaboração no frontend, proposta de contrato da API e dois ciclos TDD do backend: cálculo da idade e validação de pessoas.

Situação: Red, Green e Refactor das validações documentados; 7 testes novos e 3 anteriores passaram.
Limites: contrato da API ainda em proposta; CPF duplicado, CRUD e integração completa não verificados.
Pendência: verificação geral de formatação; apenas os arquivos deste ciclo passaram no Prettier.
Revisão: PR #6 aberto com base em backend-calculo-idade e dependente do PR #5.
Data: 20 de outubro de 2026.

ETAPA 1

Estrutura inicial

Pastas do backend, frontend, documentação e evidências foram organizadas no projeto.

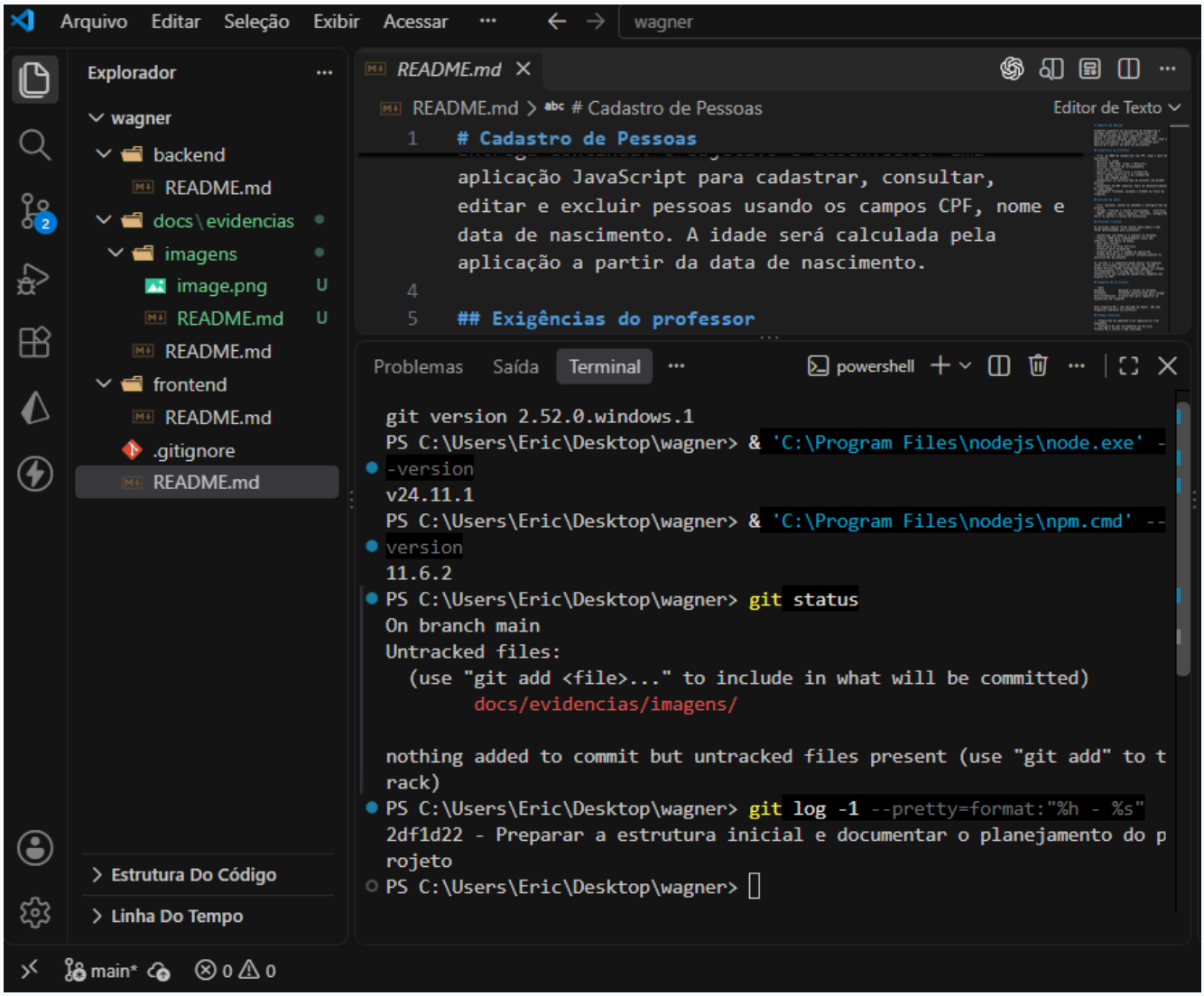


Evidência 01

ETAPA 1

Ambiente e primeiro commit

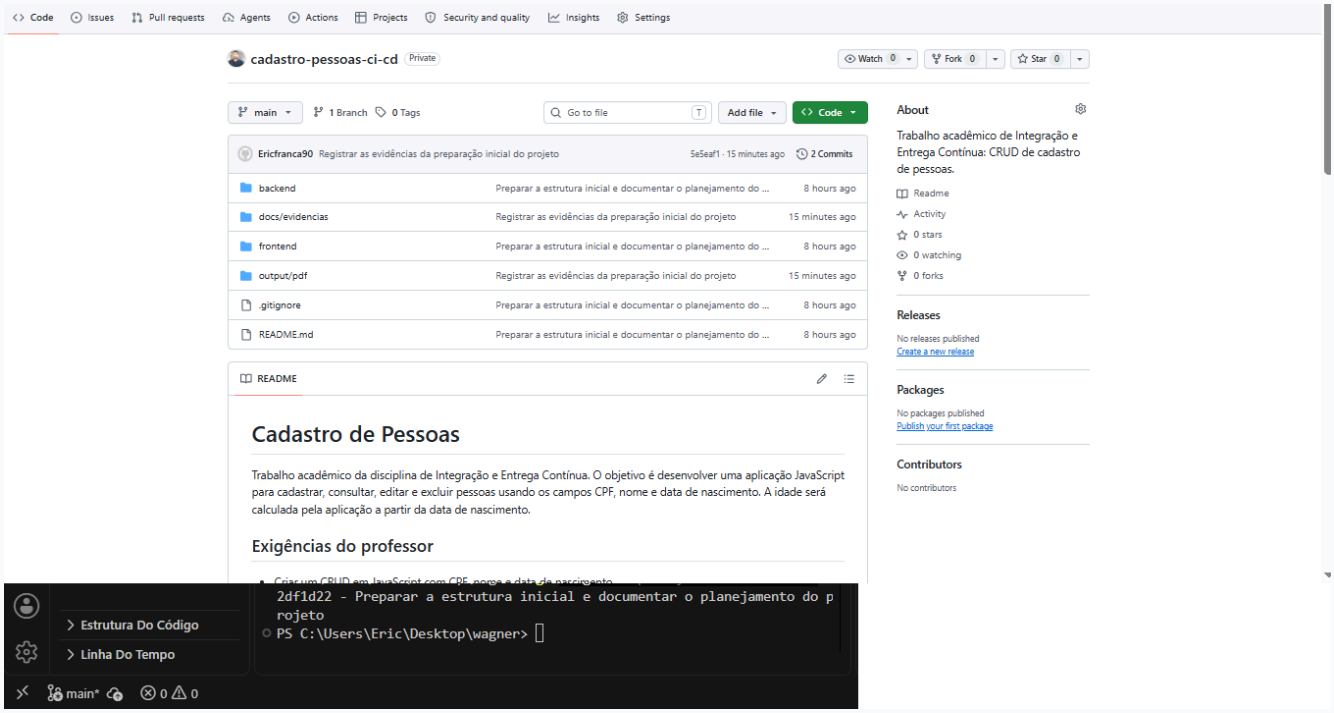
Foram conferidas as ferramentas instaladas e registrado o primeiro commit da estrutura planejada.



ETAPA 1

Repositório publicado

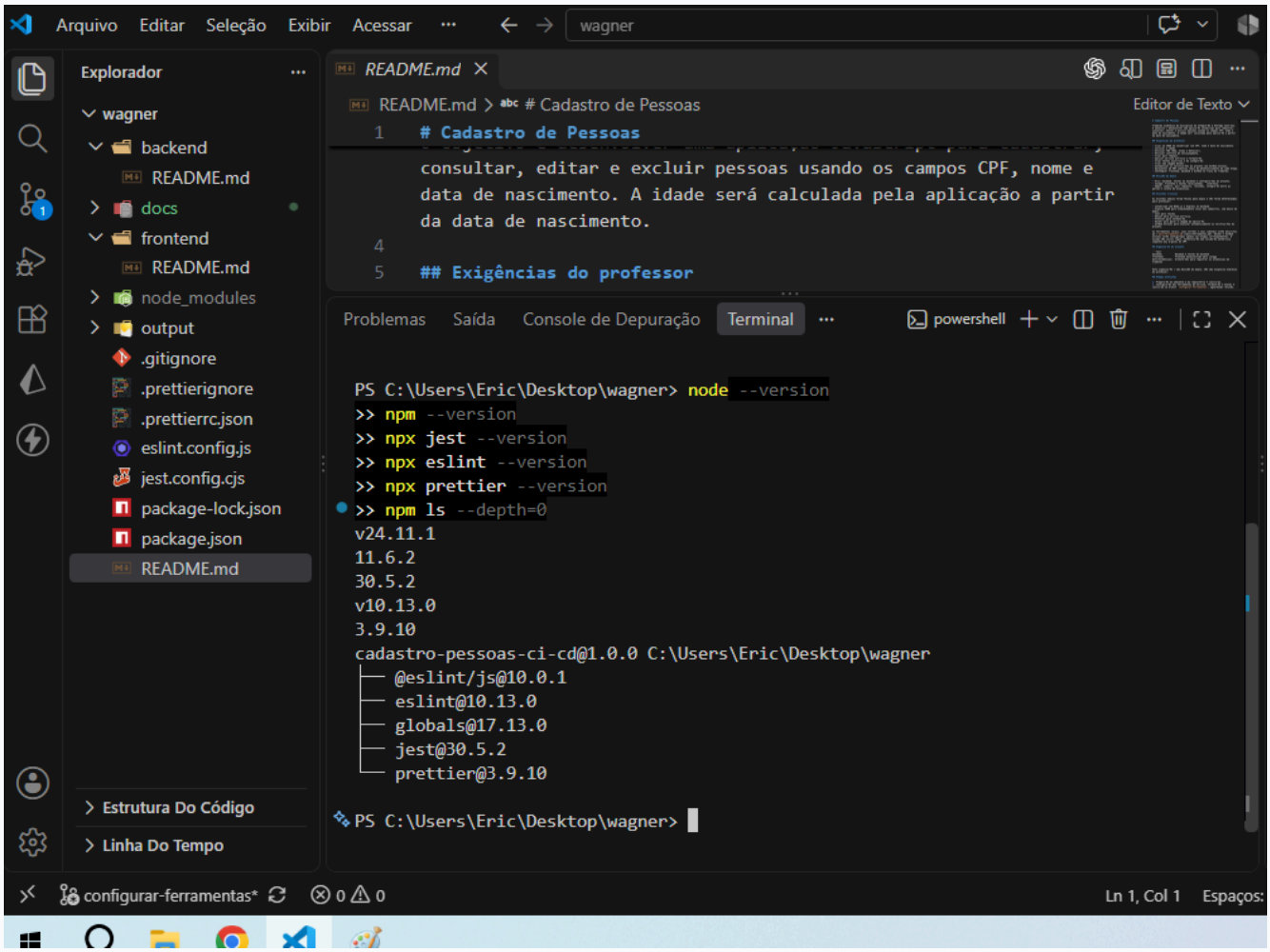
O projeto foi publicado no GitHub para permitir o trabalho em dupla e o controle de versões.



ETAPA 2

Versões e dependências

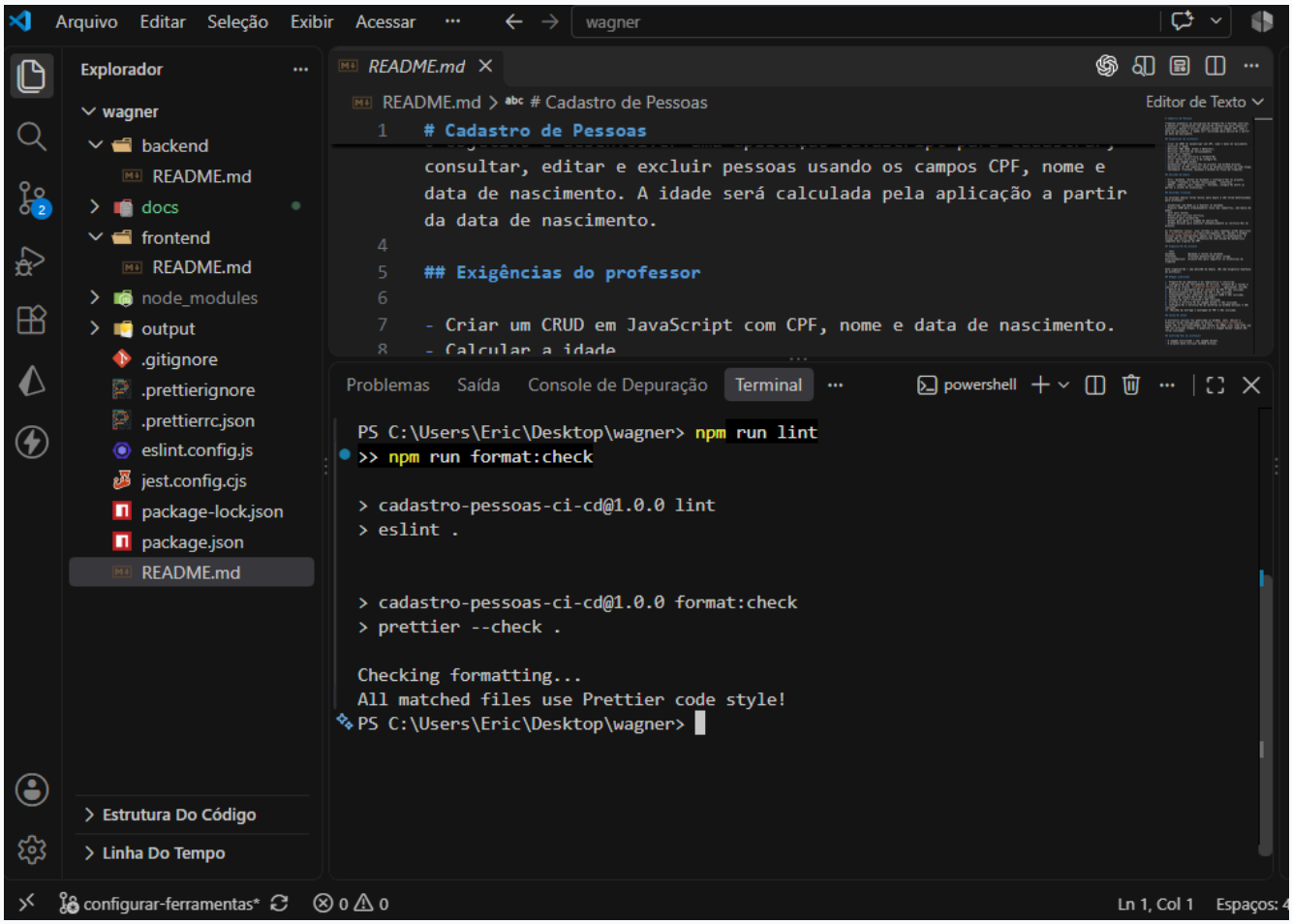
Node.js, npm, Jest, ESLint e Prettier foram conferidos, junto com as dependências instaladas.



ETAPA 2

Análise e formatação

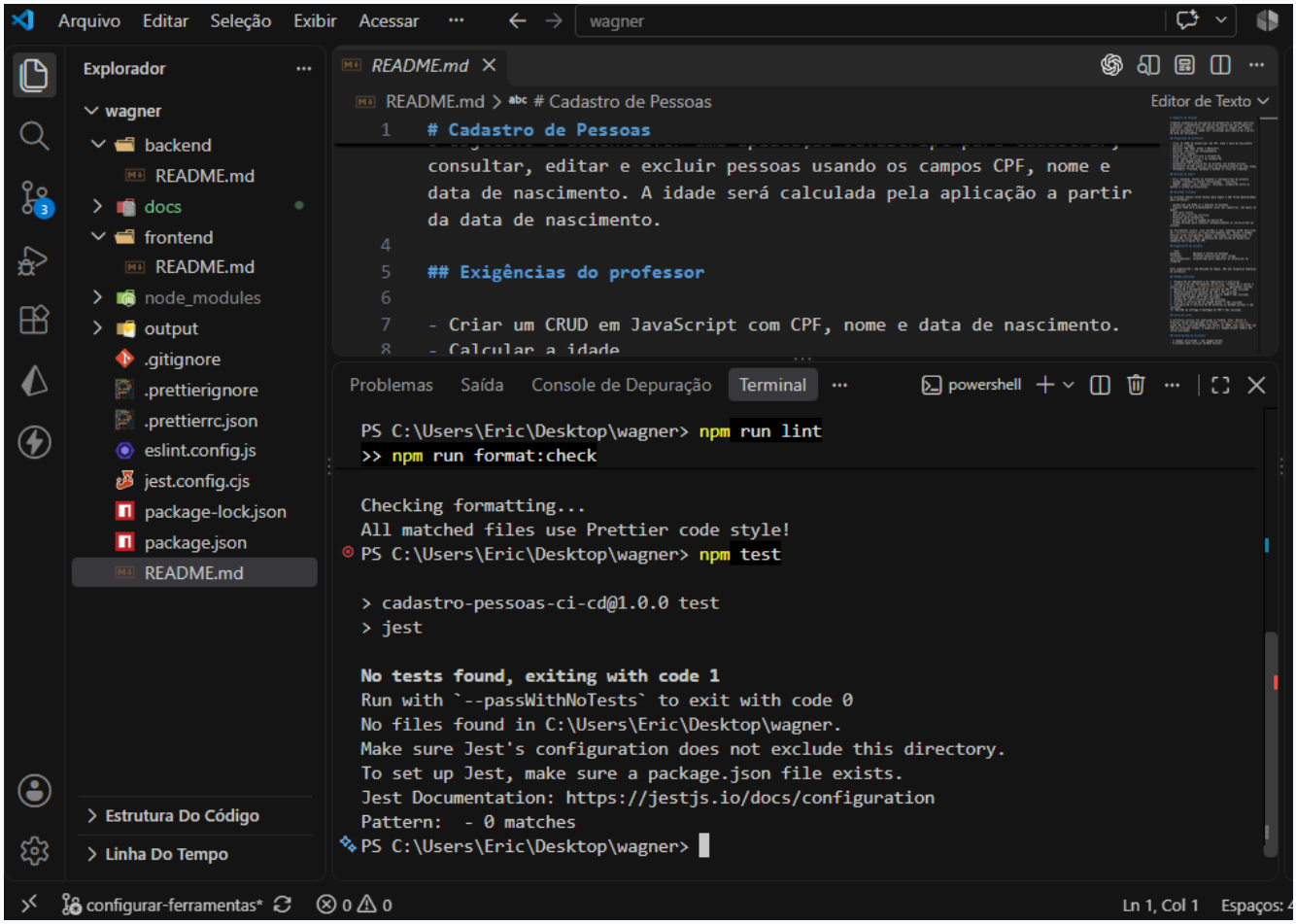
O ESLint terminou sem erros e o Prettier confirmou que os arquivos seguem a formatação definida.



ETAPA 2

Jest antes dos testes

O Jest foi executado e informou que ainda não existem testes, situação esperada antes do início do TDD.

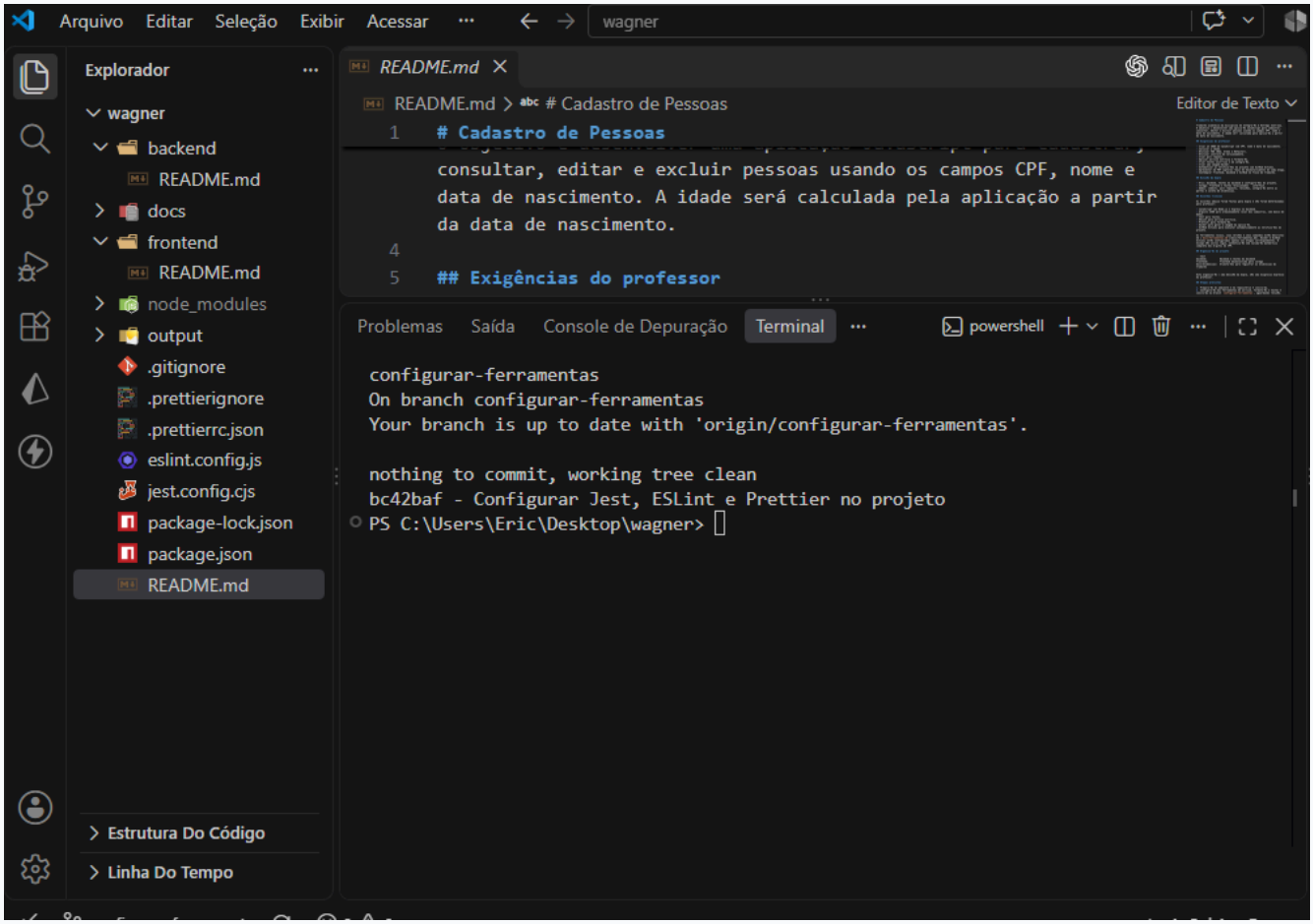


Evidência 06

ETAPA 2

Branch e commit

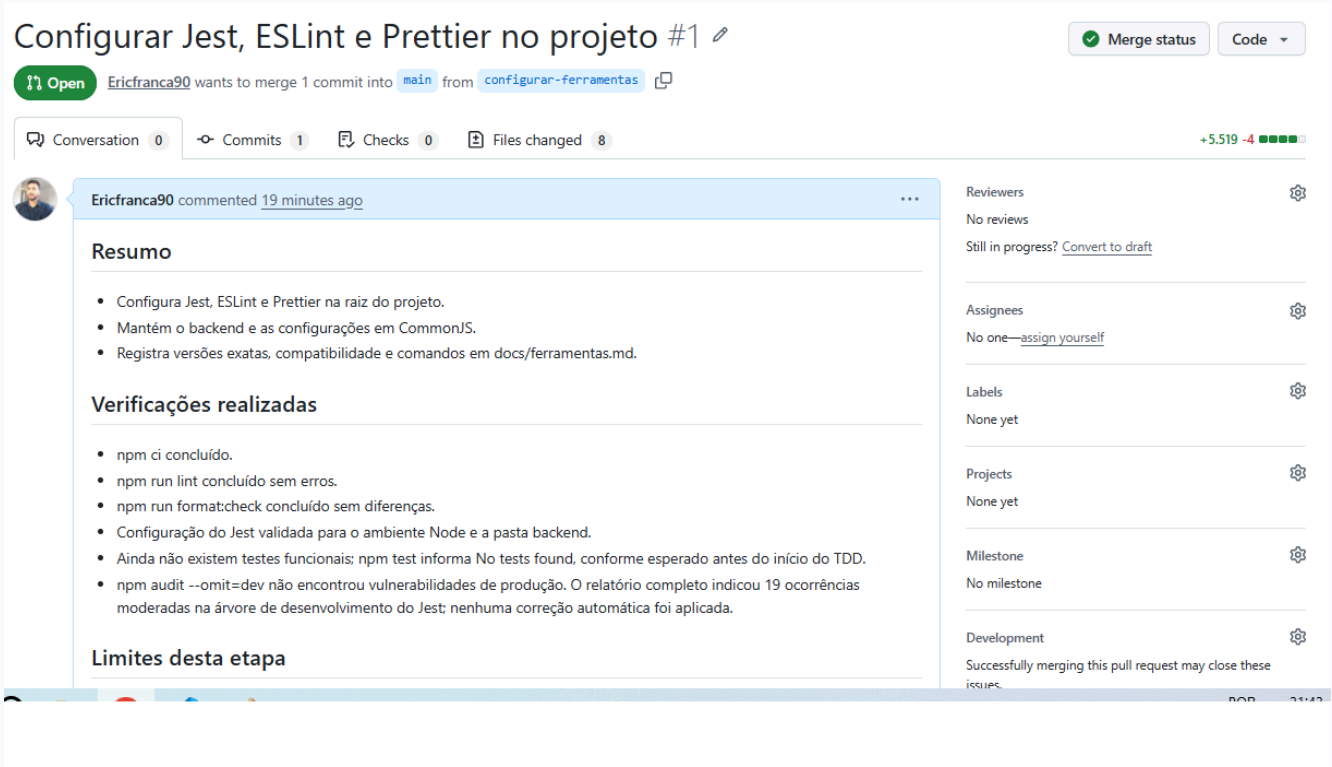
As configurações foram registradas na branch configurar-ferramentas com um commit descritivo.



ETAPA 2

Pull request aberto

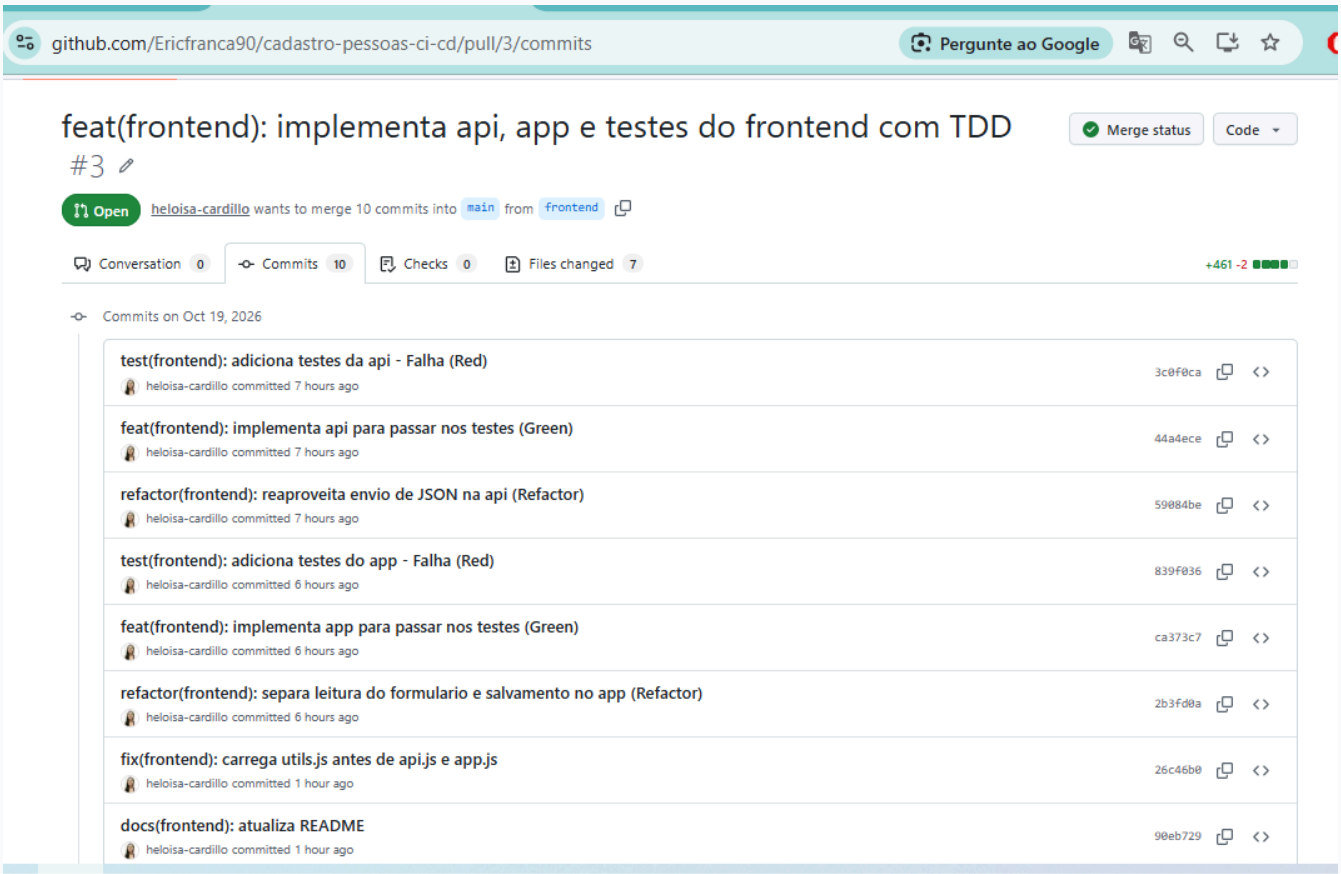
O pull request foi aberto da branch configurar-ferramentas para a main e permaneceu disponível para revisão do colega.



COLABORAÇÃO

Contribuição no frontend

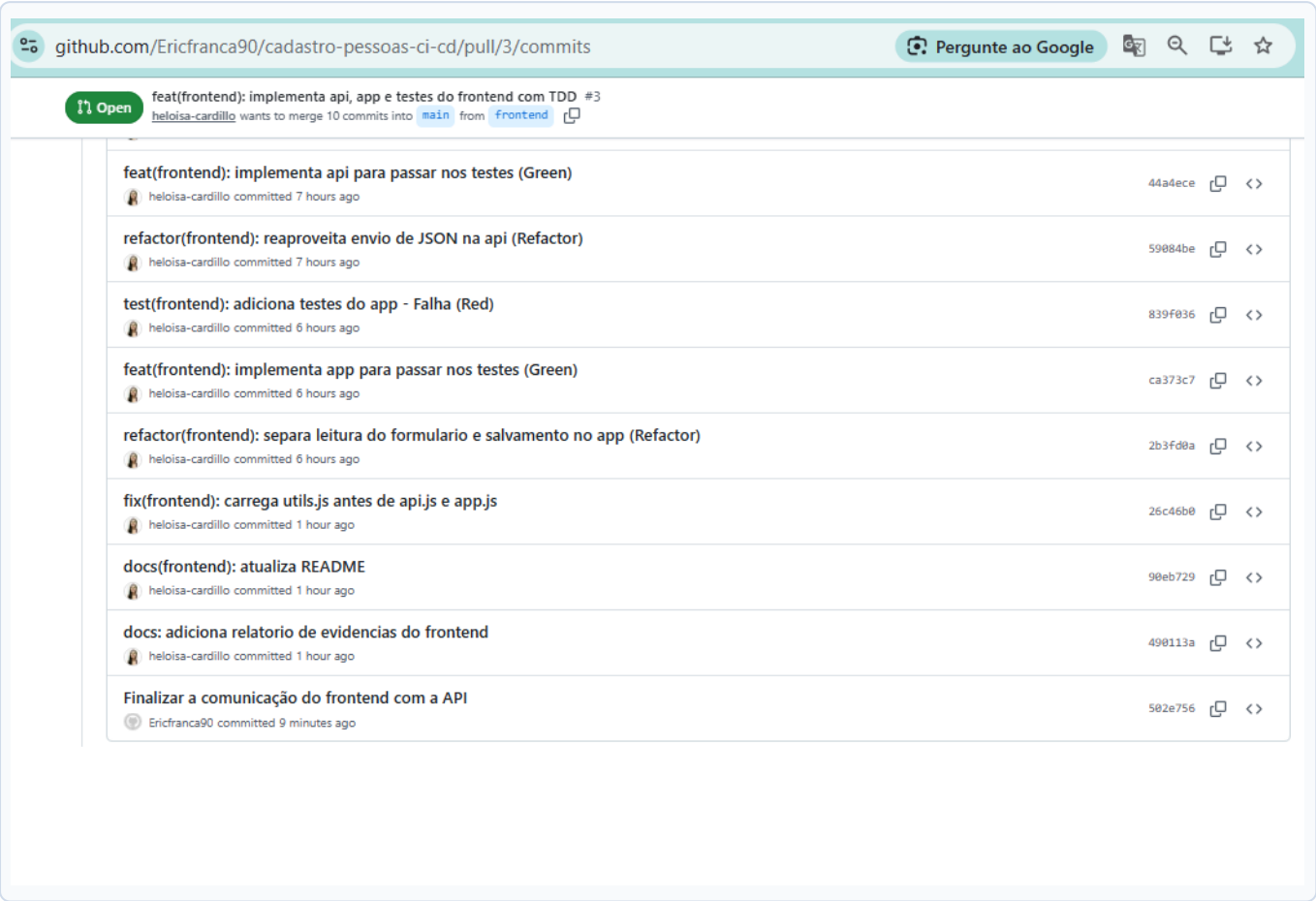
O pull request aberto mostra os commits de testes e desenvolvimento feitos pela Heloisa.



COLABORAÇÃO

Revisão em dupla

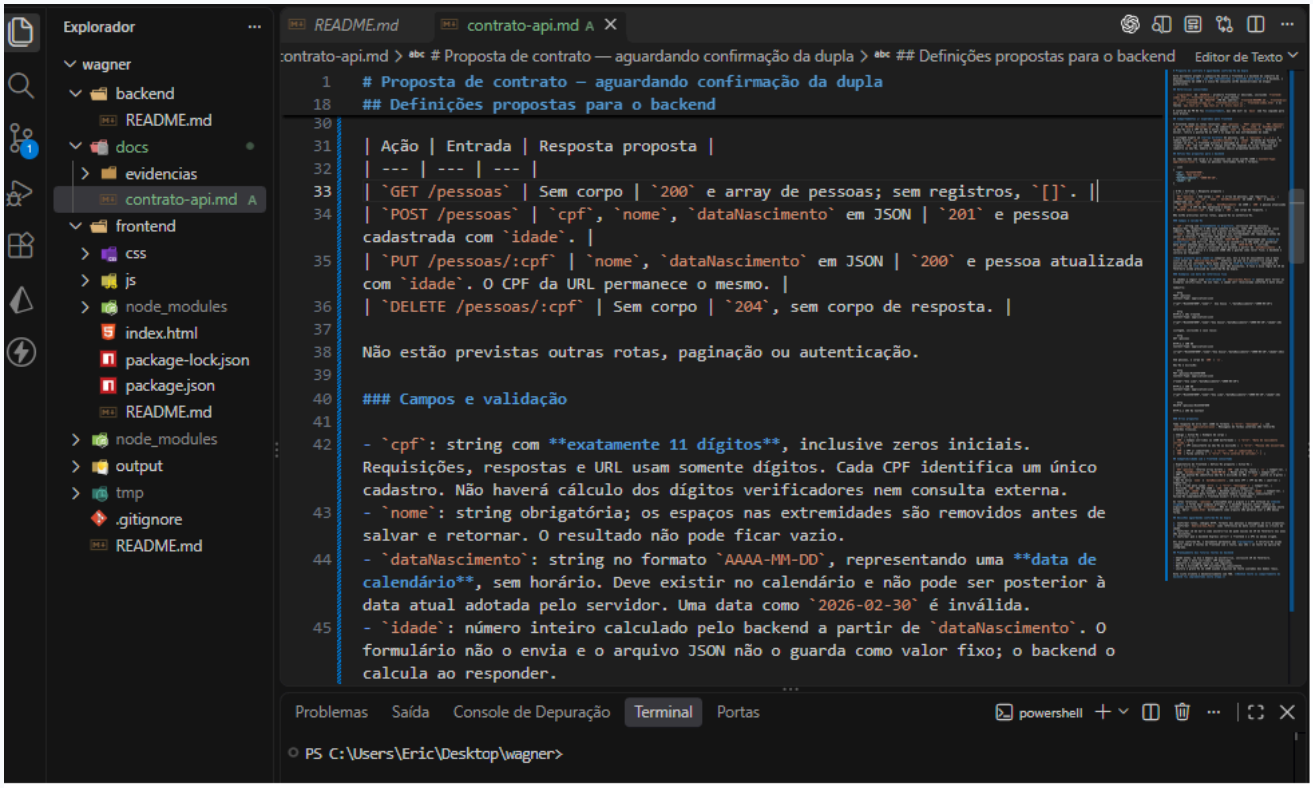
O histórico preserva os commits da Heloisa e registra a finalização feita pelo Eric.



ETAPA 3

Rotas e formatos

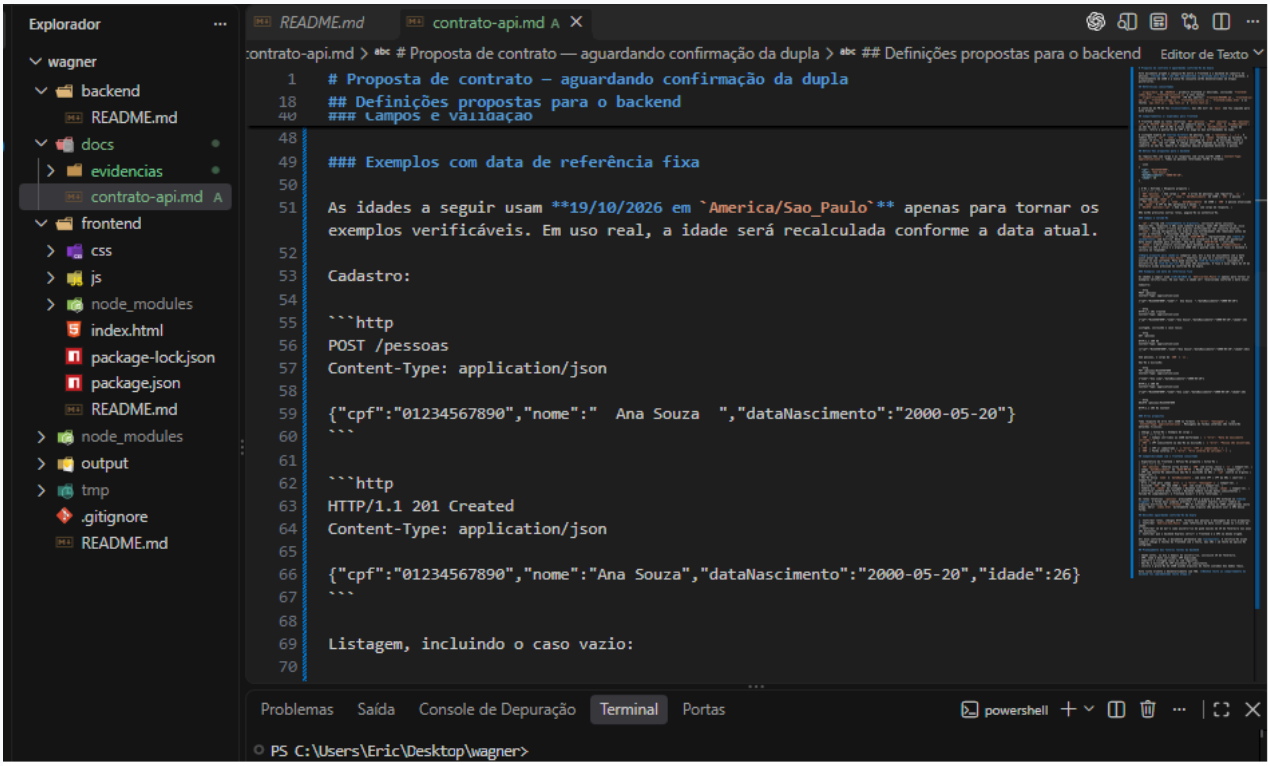
O documento propõe as quatro rotas do cadastro e os formatos dos campos.



ETAPA 3

Exemplos da API

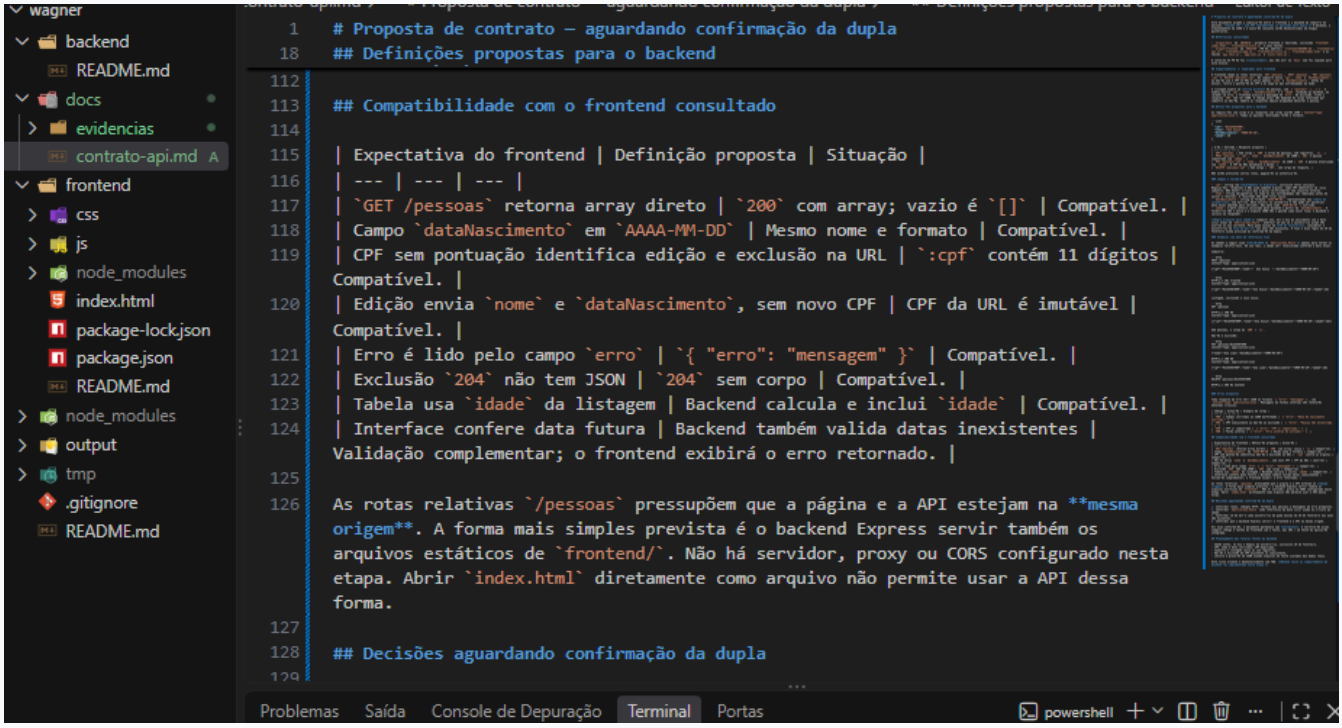
A proposta mostra um cadastro e sua resposta com a idade calculada.



ETAPA 3

Compatibilidade com o frontend

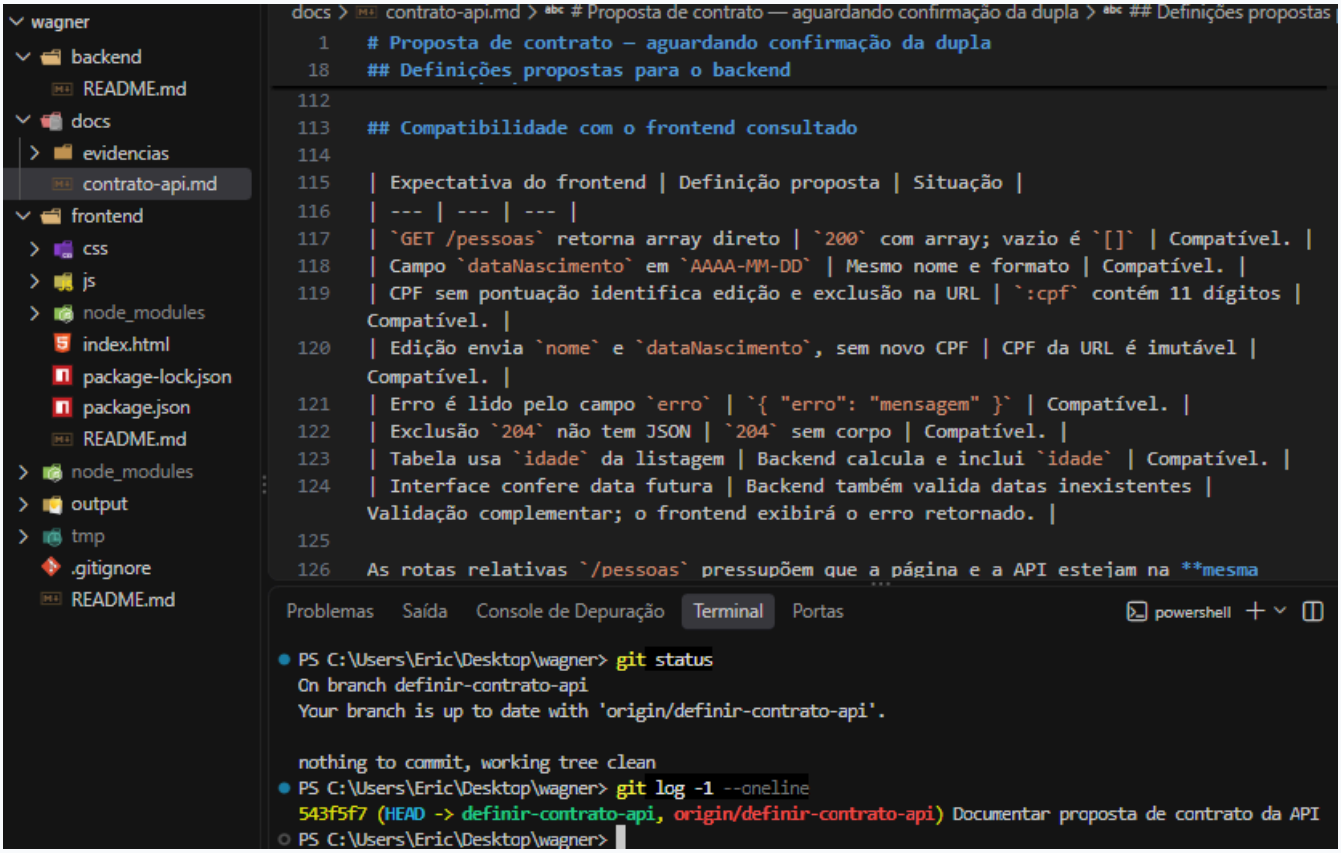
A tabela compara o contrato proposto com o código já escrito para a interface.



ETAPA 3

Branch e commit

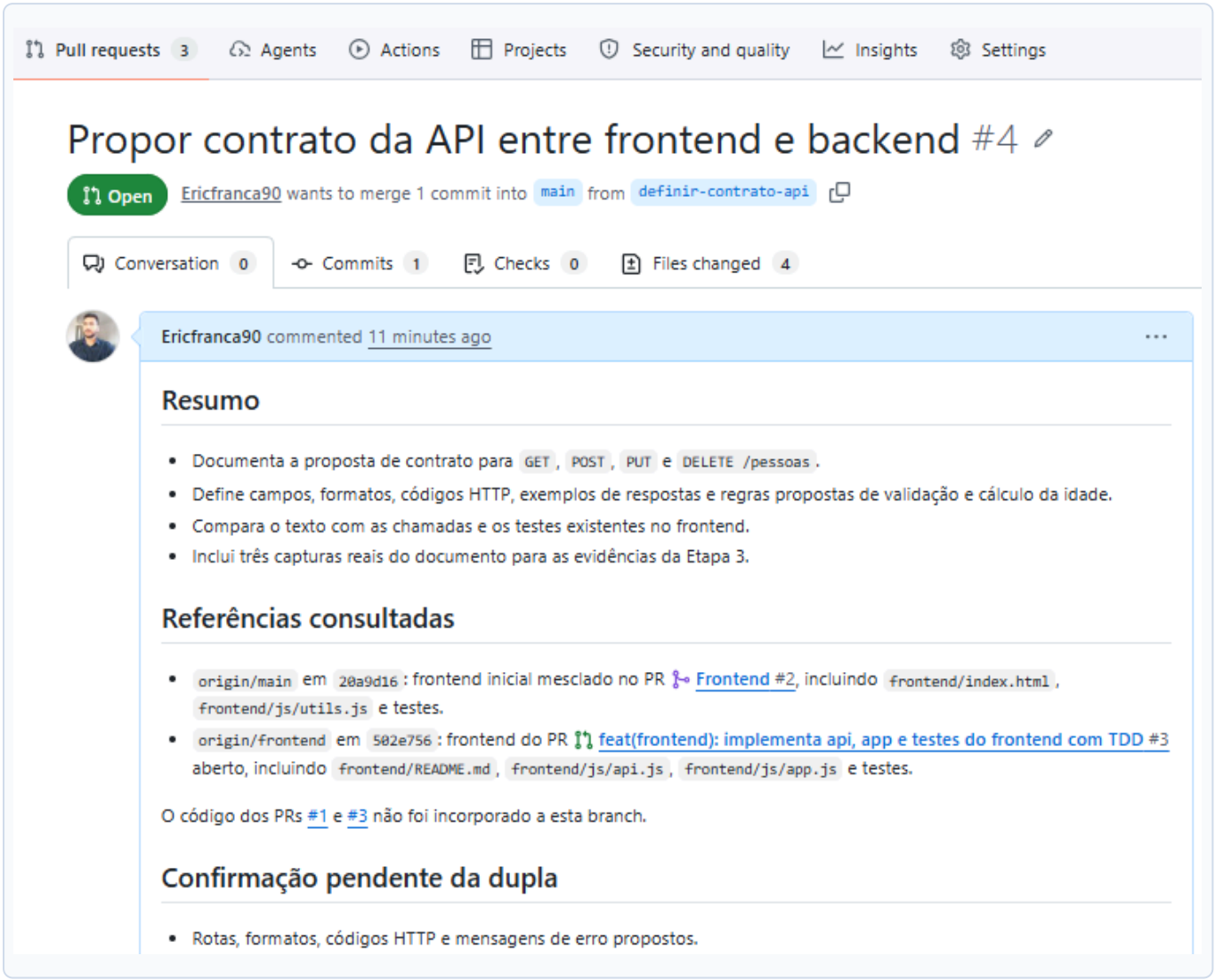
O terminal mostra a branch limpa e o commit da proposta de contrato.



ETAPA 3

Pull request para revisão

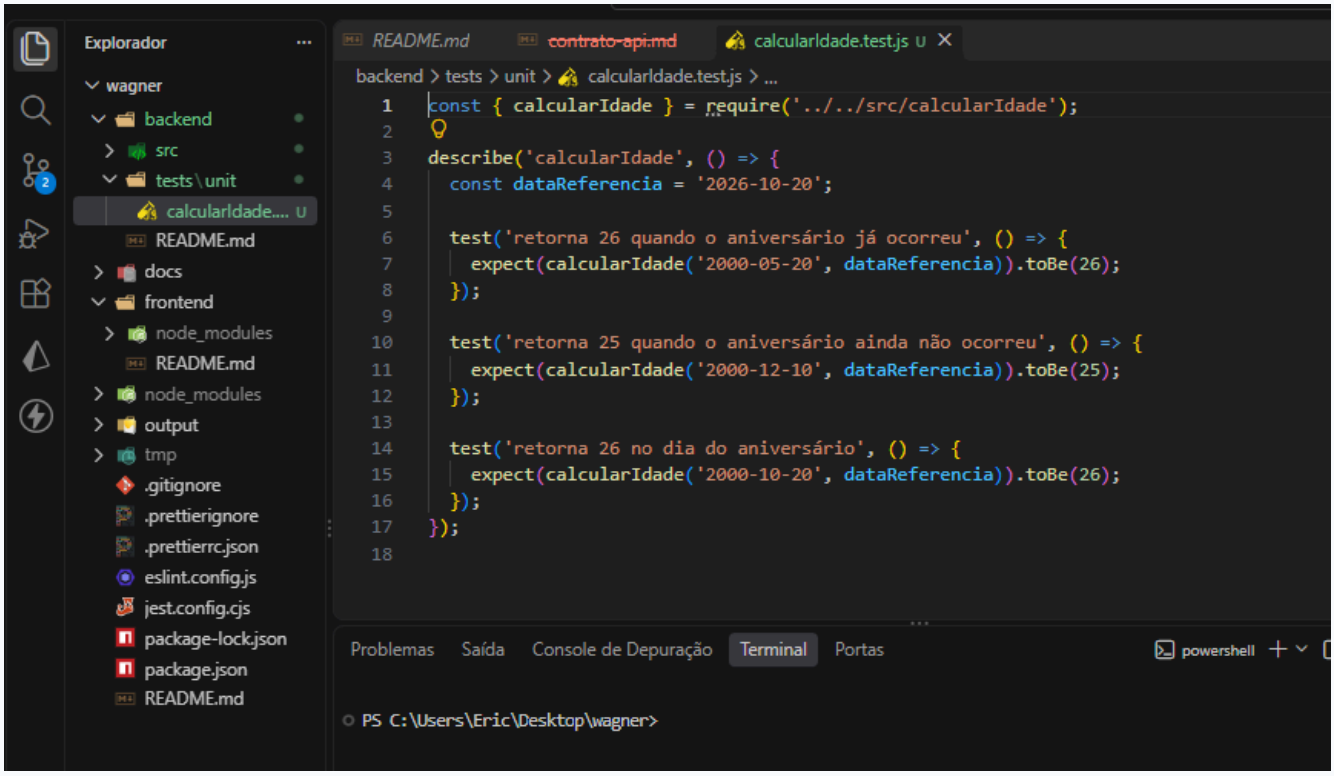
O PR #4 apresenta a proposta à dupla e permanece aberto para revisão.



ETAPA 4 - RED

Testes escritos primeiro

Três cenários usam a referência fixa de 20/10/2026; a implementação ainda não havia sido criada.



ETAPA 4 - RED

Módulo mínimo

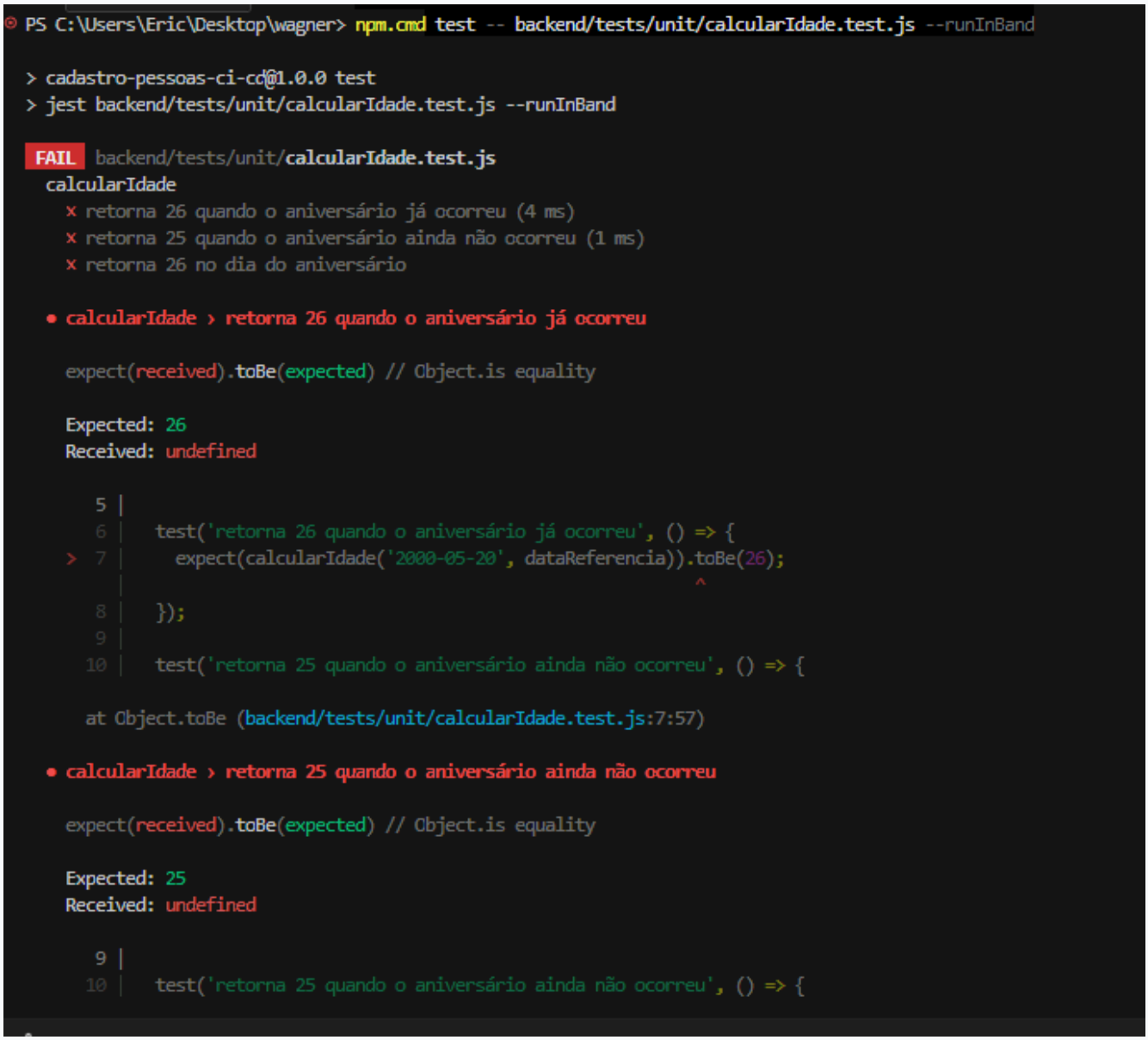
A função exportada retornava apenas undefined, sem algoritmo ou respostas preparadas.



ETAPA 4 - RED

Falha dos testes - parte 1

O Jest executou os casos; os resultados recebidos foram undefined, diferentes das idades esperadas.

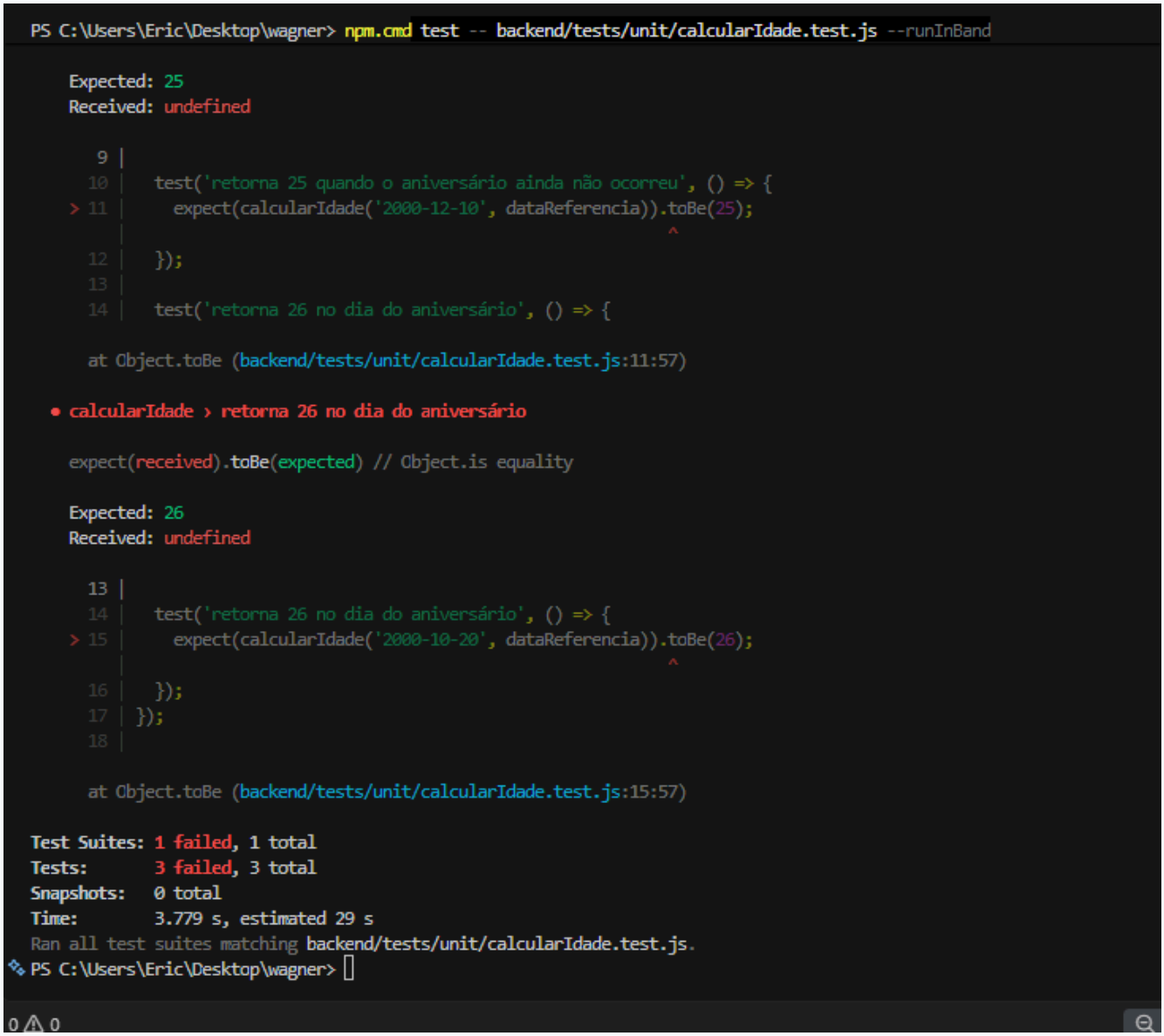


Evidência 18A

ETAPA 4 - RED

Falha dos testes - parte 2

A continuação da saída registra o terceiro caso e o resumo: 3 testes falharam.

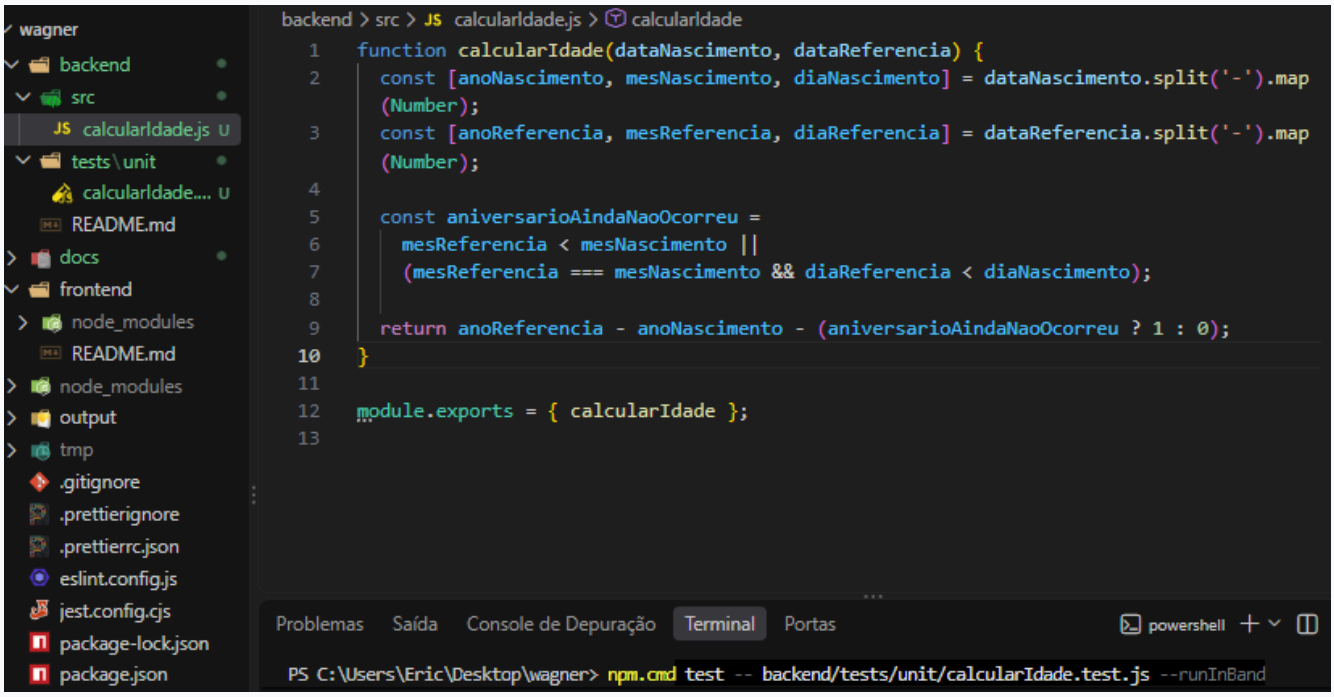


Evidência 18B

ETAPA 4 - GREEN

Cálculo implementado

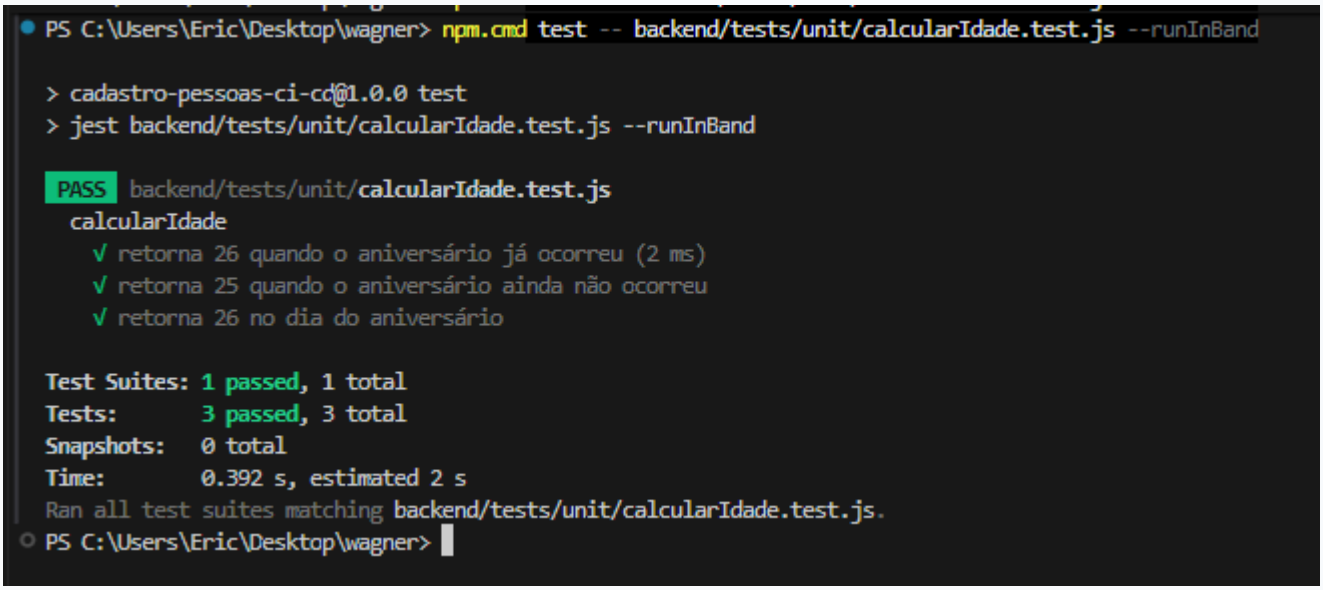
O backend compara ano, mês e dia para retornar a idade em anos completos, usando a data de referência recebida.



ETAPA 4 - GREEN

Testes passando

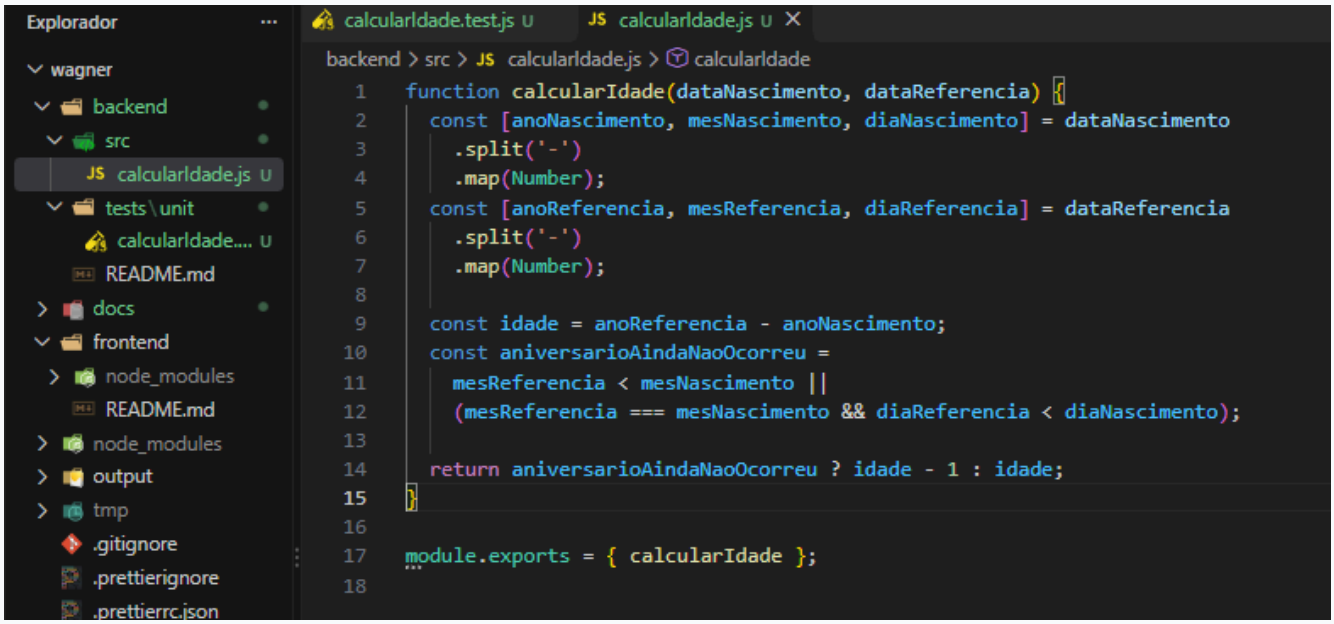
Depois da implementação, os três testes unitários passaram.



ETAPA 4 - REFACTOR

Código reorganizado

A idade-base foi separada da regra de aniversário ainda não ocorrido, sem acrescentar comportamento.



ETAPA 4 - REFACTOR

Resultados preservados

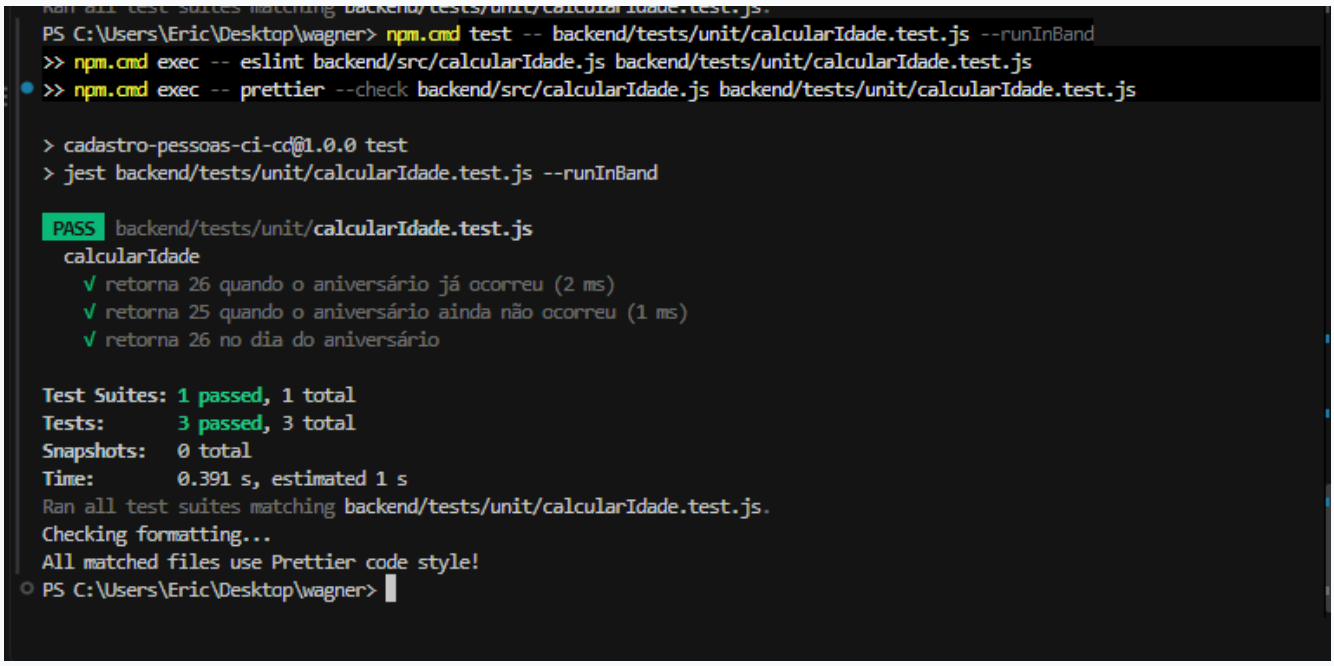
Após a refatoração, os mesmos três testes continuaram passando.



ETAPA 4 - VERIFICAÇÕES

Testes e análise dos arquivos novos

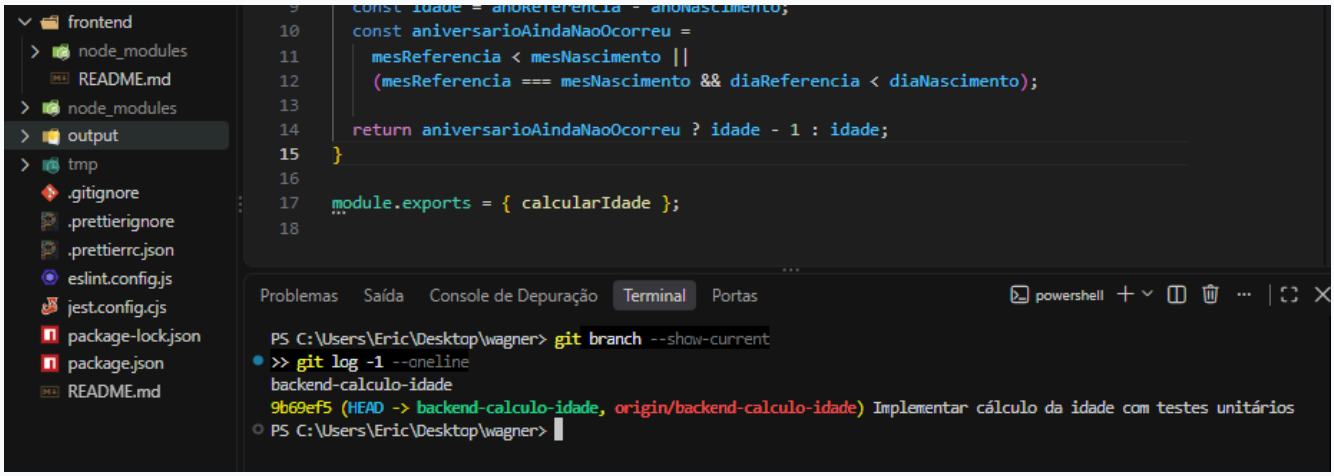
Os três testes, o ESLint e o Prettier dos dois arquivos deste ciclo passaram. A verificação geral de formatação continua pendente.



ETAPA 4 - GIT

Branch e commit de código

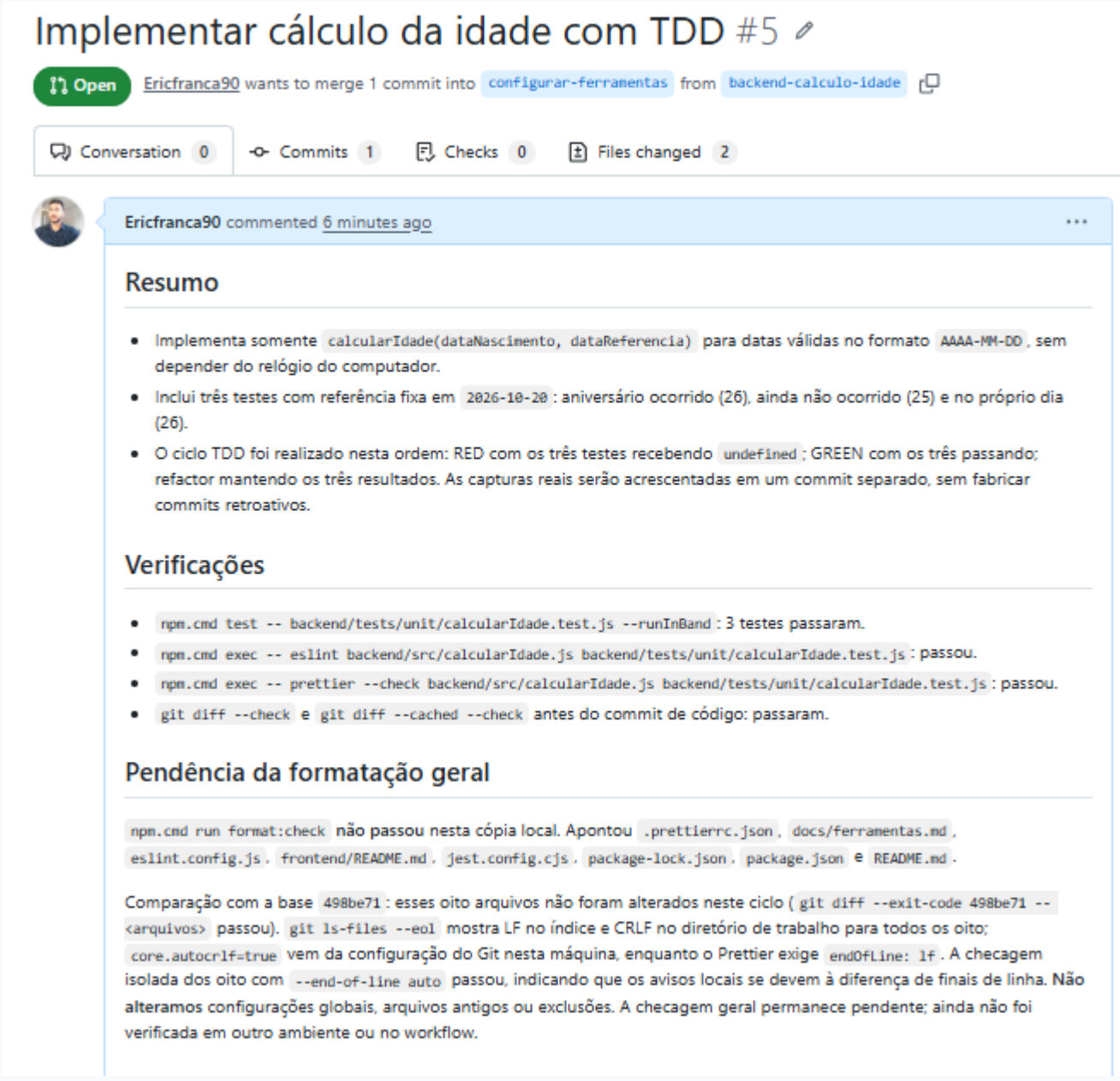
O código e os testes foram registrados juntos, sem criar commits retroativos das fases TDD.



ETAPA 4 - GIT

Pull request para revisão

O PR #5 permanece aberto, da branch backend-calculo-idade para configurar-ferramentas, da qual depende.



Implementar cálculo da idade com TDD #5

Open Ericfranca90 wants to merge 1 commit into `configurar-ferramentas` from `backend-calculo-idade`

Conversation 0 · Commits 1 · Checks 0 · Files changed 2

Ericfranca90 commented 6 minutes ago

Resumo

- Implementa somente `calcularIdade(dataNascimento, dataReferencia)` para datas válidas no formato `AAAA-MM-DD`, sem depender do relógio do computador.
- Inclui três testes com referência fixa em `2026-10-20`: aniversário ocorrido (26), ainda não ocorrido (25) e no próprio dia (26).
- O ciclo TDD foi realizado nesta ordem: RED com os três testes recebendo `undefined`; GREEN com os três passando; refactor mantendo os três resultados. As capturas reais serão acrescentadas em um commit separado, sem fabricar commits retroativos.

Verificações

- `npm.cmd test -- backend/tests/unit/calcularIdade.test.js --runInBand`: 3 testes passaram.
- `npm.cmd exec -- eslint backend/src/calcularIdade.js backend/tests/unit/calcularIdade.test.js`: passou.
- `npm.cmd exec -- prettier --check backend/src/calcularIdade.js backend/tests/unit/calcularIdade.test.js`: passou.
- `git diff --check` e `git diff --cached --check` antes do commit de código: passaram.

Pendência da formatação geral

`npm.cmd run format:check` não passou nesta cópia local. Apontou `.prettierrc.json`, `docs/ferramentas.md`, `eslint.config.js`, `frontend/README.md`, `jest.config.cjs`, `package-lock.json`, `package.json` e `README.md`.

Comparação com a base `498be71`: esses oito arquivos não foram alterados neste ciclo (`git diff --exit-code 498be71 -- <arquivos>` passou). `git ls-files --eol` mostra LF no índice e CRLF no diretório de trabalho para todos os oito; `core.autocrlf=true` vem da configuração do Git nesta máquina, enquanto o Prettier exige `endOfLine: lf`. A checagem isolada dos oito com `--end-of-line auto` passou, indicando que os avisos locais se devem à diferença de finais de linha. Não alteramos configurações globais, arquivos antigos ou exclusões. A checagem geral permanece pendente; ainda não foi verificada em outro ambiente ou no workflow.

ETAPA 5 - RED

Testes escritos primeiro

Sete cenários usam dados fictícios e a referência fixa de 20/10/2026.



```
const { validarPessoa } = require('../../src/validarPessoa');

describe('validarPessoa', () => {
  const dataReferencia = '2026-10-20';

  function pessoaValida() {
    return {
      cpf: '01234567890',
      nome: 'Ana Souza',
      dataNascimento: '2000-05-20',
    };
  }

  test('retorna null para um cadastro válido', () => {
    expect(validarPessoa(pessoaValida(), dataReferencia)).toBeNull();
  });

  test('rejeita CPF com menos de 11 dígitos', () => {
    const dados = { ...pessoaValida(), cpf: '0123456789' };
    expect(validarPessoa(dados, dataReferencia)).toBe('CPF deve ter 11 dígitos.');
  });

  test('rejeita CPF com 11 caracteres contendo letra', () => {
    const dados = { ...pessoaValida(), cpf: '0123456789A' };
    expect(validarPessoa(dados, dataReferencia)).toBe('CPF deve ter 11 dígitos.');
  });

  test('rejeita nome contendo somente espaços', () => {
    const dados = { ...pessoaValida(), nome: '   ' };
    expect(validarPessoa(dados, dataReferencia)).toBe('Informe o nome.');
  });

  test('rejeita data de nascimento vazia', () => {
    const dados = { ...pessoaValida(), dataNascimento: '' };
    expect(validarPessoa(dados, dataReferencia)).toBe('Informe a data de nascimento.');
  });

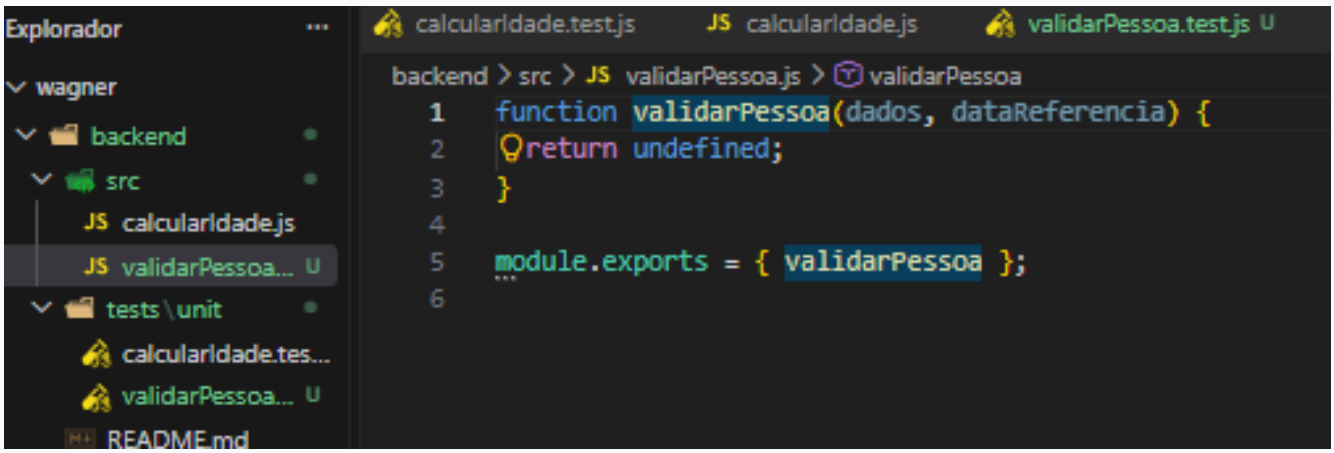
  test('rejeita data de nascimento inexistente', () => {
    const dados = { ...pessoaValida(), dataNascimento: '2026-02-30' };
    expect(validarPessoa(dados, dataReferencia)).toBe('Data de nascimento inválida.');
  });

  test('rejeita nascimento posterior à data de referência', () => {
    const dados = { ...pessoaValida(), dataNascimento: '2026-10-21' };
    expect(validarPessoa(dados, dataReferencia)).toBe(
      'A data de nascimento não pode ser no futuro.',
    );
  });
});
```

ETAPA 5 - RED

Módulo mínimo

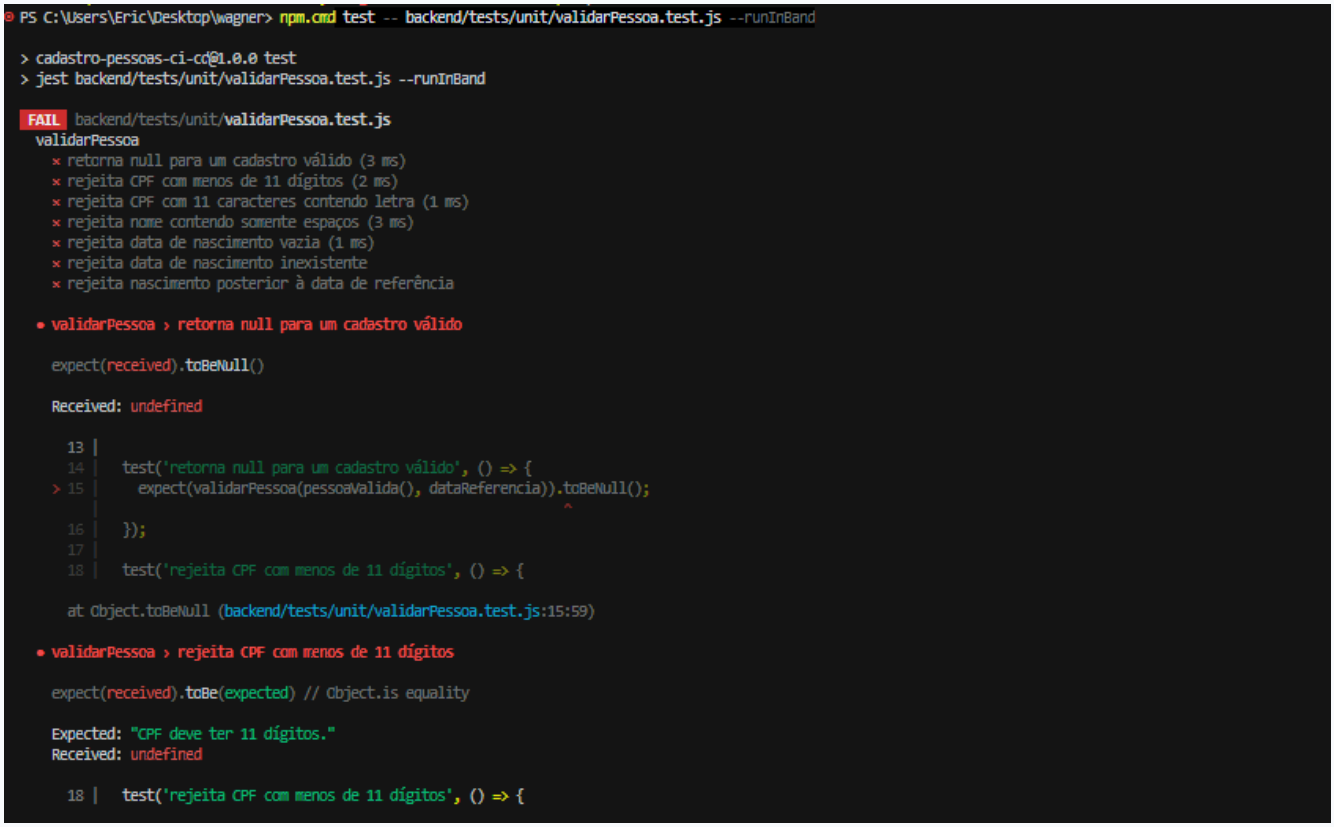
A função exportada ainda retornava undefined, sem validar qualquer campo.



ETAPA 5 - RED

Falhas dos testes - parte 1

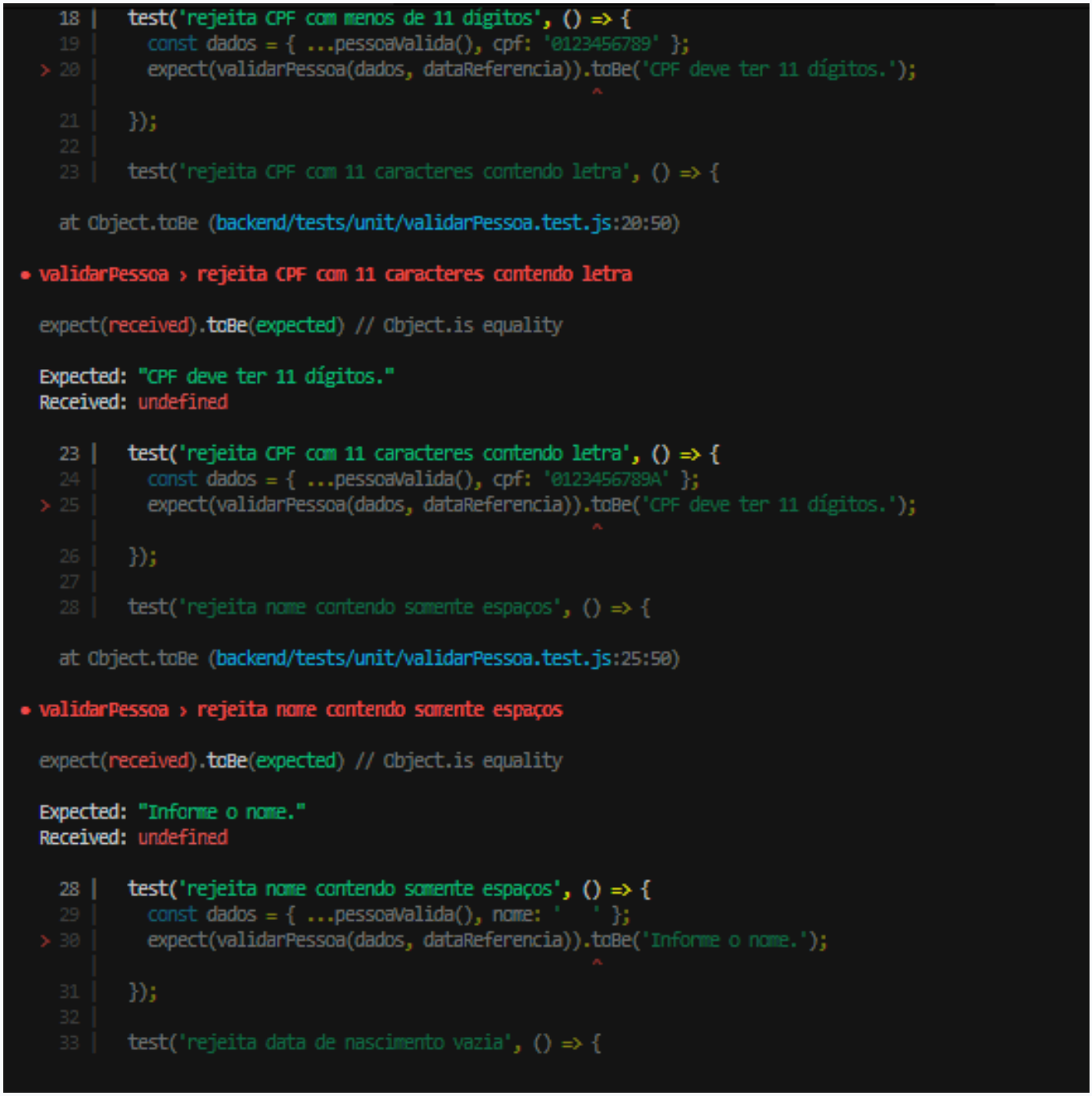
O Jest executou os sete testes; as primeiras falhas mostram o resultado undefined.



ETAPA 5 - RED

Falhas dos testes - parte 2

A continuação registra a rejeição esperada para CPF e nome inválidos.



```
  18 |   test('rejeita CPF com menos de 11 dígitos', () => {
  19 |     const dados = { ...pessoaValida(), cpf: '0123456789' };
> 20 |     expect(validarPessoa(dados, dataReferencia)).toBe('CPF deve ter 11 dígitos.');
     |                                                  ^
  21 |   });
  22 |
  23 |   test('rejeita CPF com 11 caracteres contendo letra', () => {

  at Object.toBe (backend/tests/unit/validarPessoa.test.js:20:50)

● validarPessoa › rejeita CPF com 11 caracteres contendo letra

  expect(received).toBe(expected) // Object.is equality

  Expected: "CPF deve ter 11 dígitos."
  Received: undefined

  23 |   test('rejeita CPF com 11 caracteres contendo letra', () => {
  24 |     const dados = { ...pessoaValida(), cpf: '0123456789A' };
> 25 |     expect(validarPessoa(dados, dataReferencia)).toBe('CPF deve ter 11 dígitos.');
     |                                                  ^
  26 |   });
  27 |
  28 |   test('rejeita nome contendo somente espaços', () => {

  at Object.toBe (backend/tests/unit/validarPessoa.test.js:25:50)

● validarPessoa › rejeita nome contendo somente espaços

  expect(received).toBe(expected) // Object.is equality

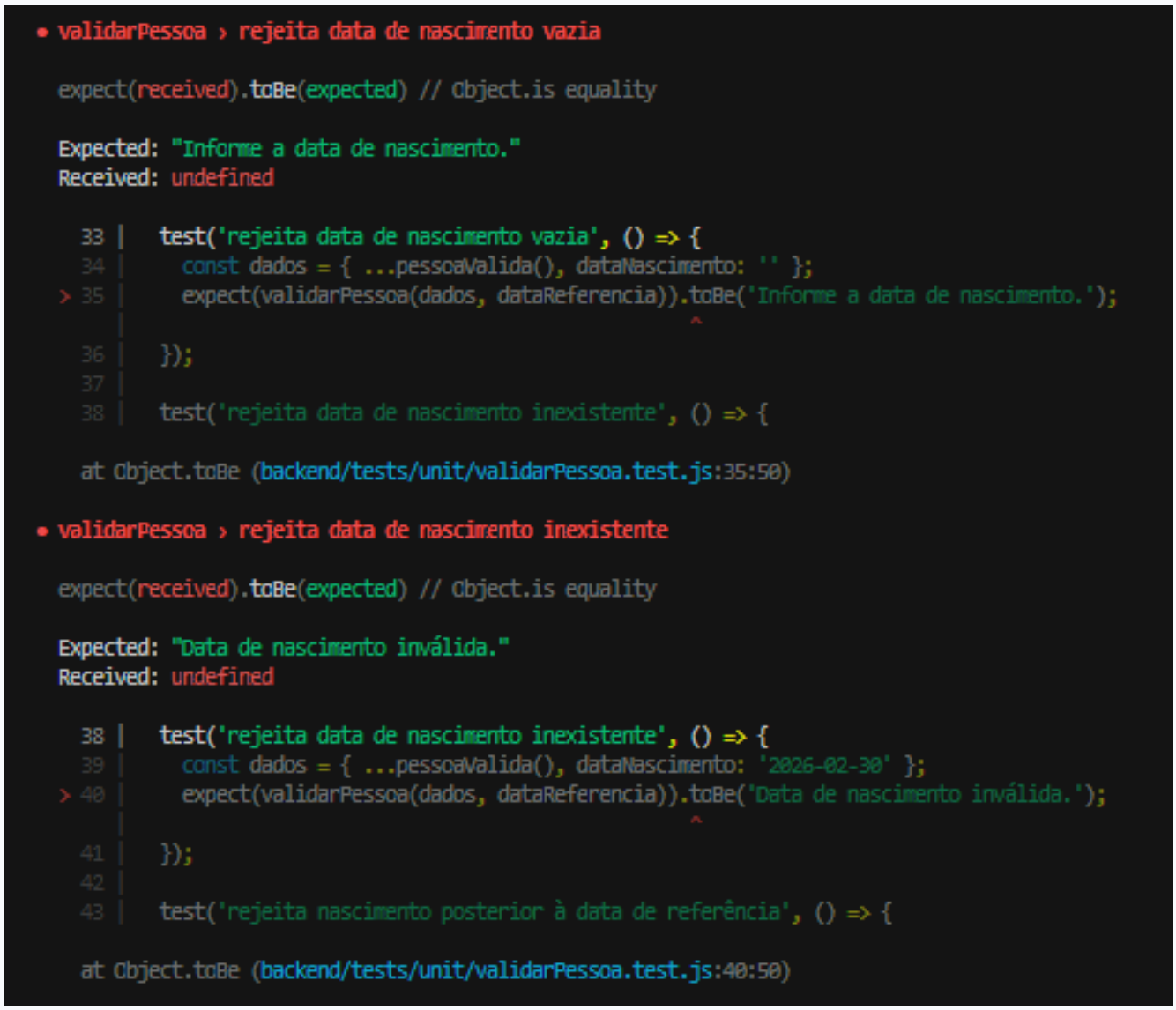
  Expected: "Informe o nome."
  Received: undefined

  28 |   test('rejeita nome contendo somente espaços', () => {
  29 |     const dados = { ...pessoaValida(), nome: '   ' };
> 30 |     expect(validarPessoa(dados, dataReferencia)).toBe('Informe o nome.');
     |                                                  ^
  31 |   });
  32 |
  33 |   test('rejeita data de nascimento vazia', () => {
```

ETAPA 5 - RED

Falhas dos testes - parte 3

Os casos de nascimento vazio e data inexistente também falharam por undefined.



```
● validarPessoa › rejeita data de nascimento vazia

  expect(received).toBe(expected) // Object.is equality

  Expected: "Informe a data de nascimento."
  Received: undefined

    33 |   test('rejeita data de nascimento vazia', () => {
    34 |     const dados = { ...pessoaValida(), dataNascimento: '' };
  > 35 |     expect(validarPessoa(dados, dataReferencia)).toBe('Informe a data de nascimento.');
       |                                                  ^
    36 |   });
    37 |
    38 |   test('rejeita data de nascimento inexistente', () => {

    at Object.toBe (backend/tests/unit/validarPessoa.test.js:35:50)

● validarPessoa › rejeita data de nascimento inexistente

  expect(received).toBe(expected) // Object.is equality

  Expected: "Data de nascimento inválida."
  Received: undefined

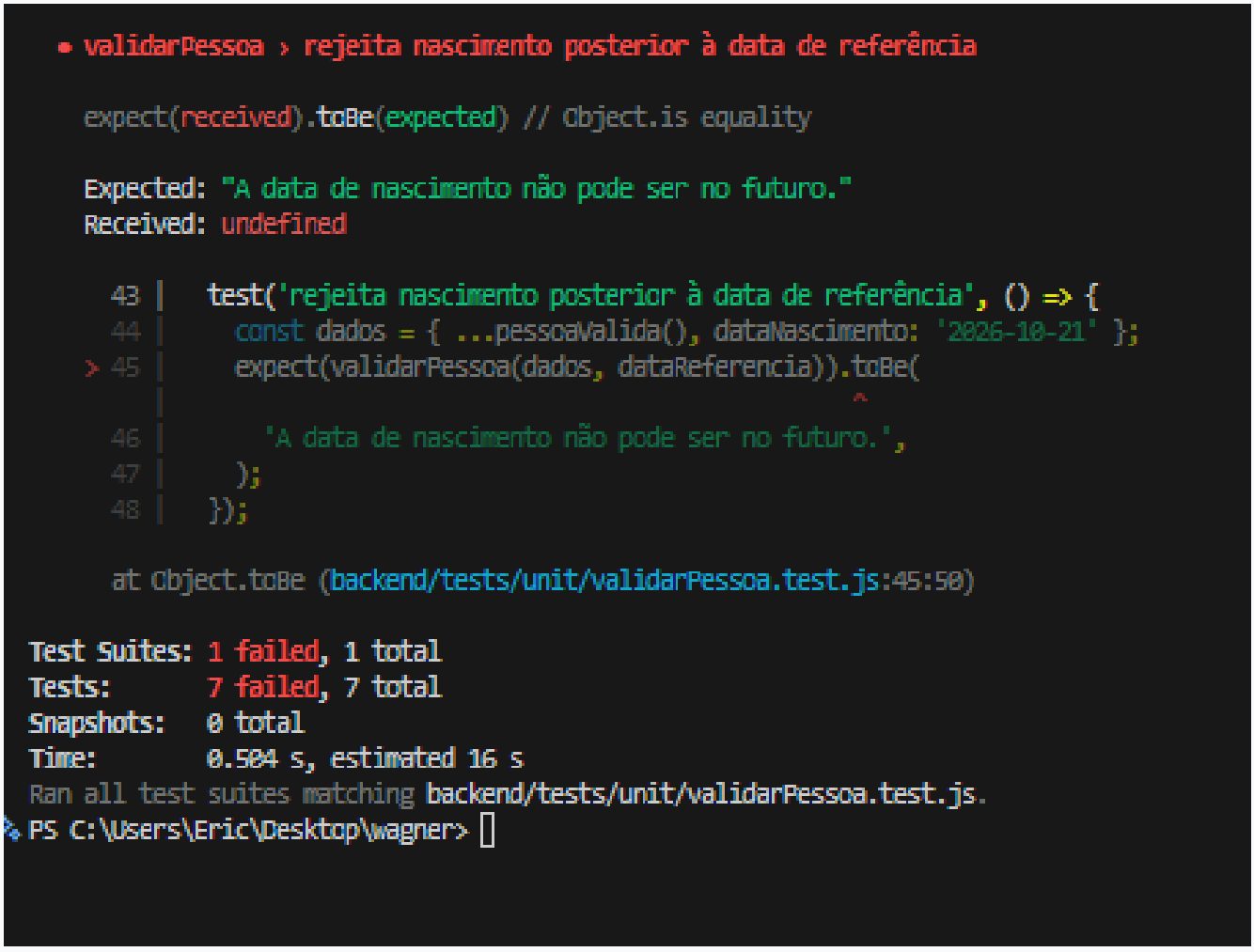
    38 |   test('rejeita data de nascimento inexistente', () => {
    39 |     const dados = { ...pessoaValida(), dataNascimento: '2026-02-30' };
  > 40 |     expect(validarPessoa(dados, dataReferencia)).toBe('Data de nascimento inválida.');
       |                                                  ^
    41 |   });
    42 |
    43 |   test('rejeita nascimento posterior à data de referência', () => {

    at Object.toBe (backend/tests/unit/validarPessoa.test.js:40:50)
```

ETAPA 5 - RED

Falhas dos testes - parte 4

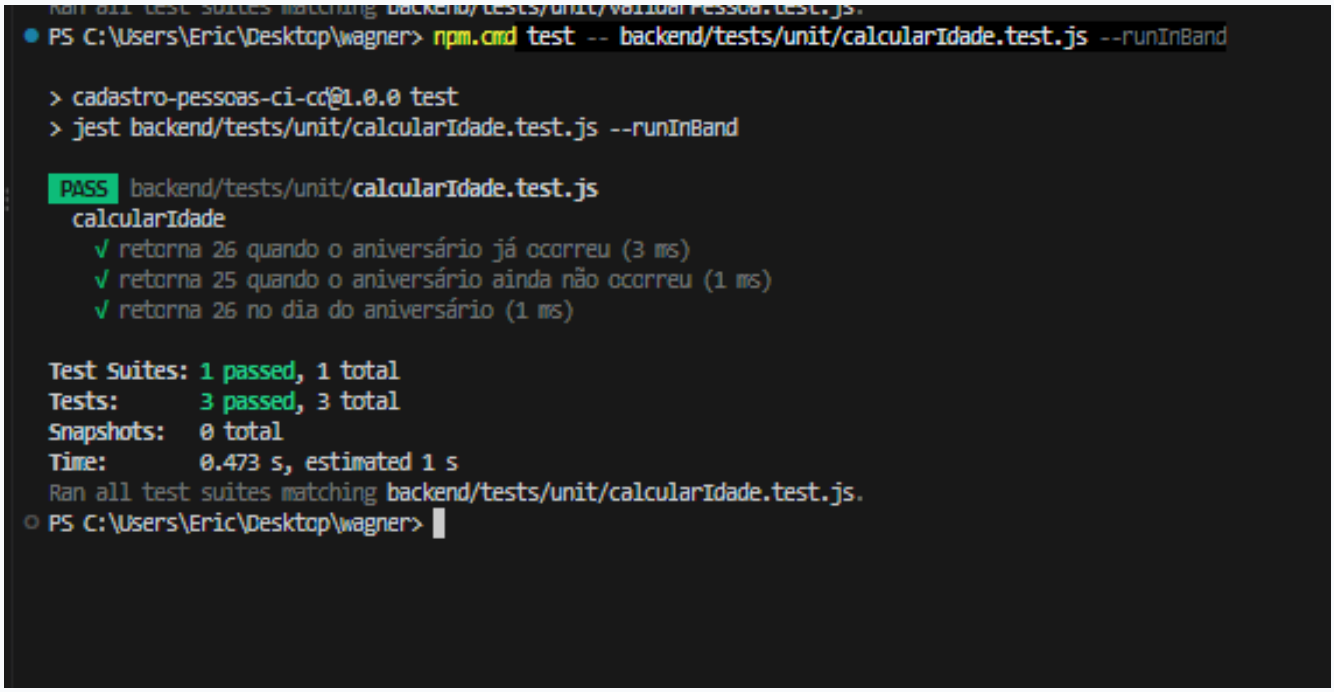
O caso de data futura falhou, e o resumo confirma sete testes falhando.



ETAPA 5 - RED

Testes anteriores preservados

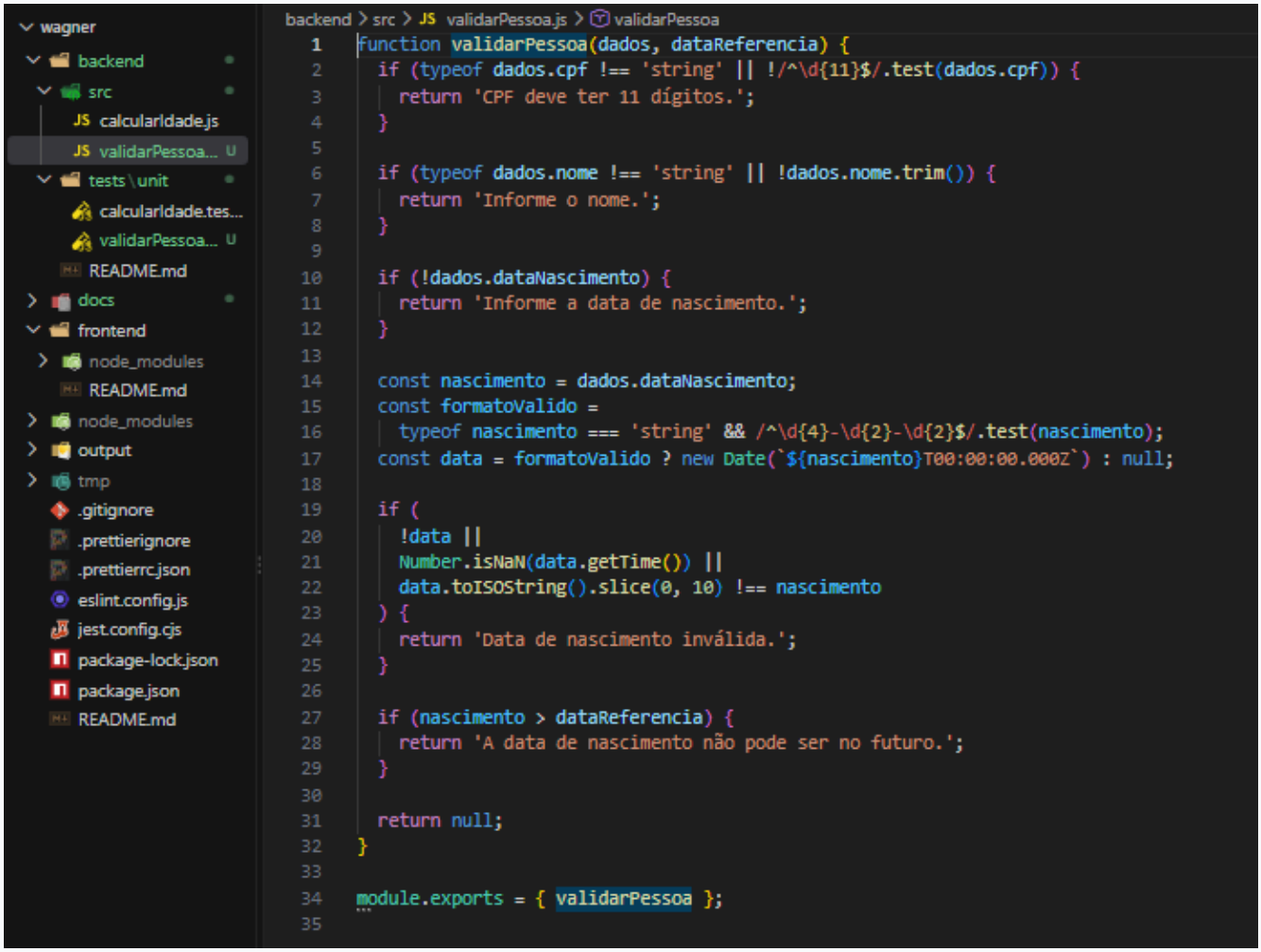
Os três testes do cálculo da idade continuaram passando durante o RED das validações.



ETAPA 5 - GREEN

Validação implementada

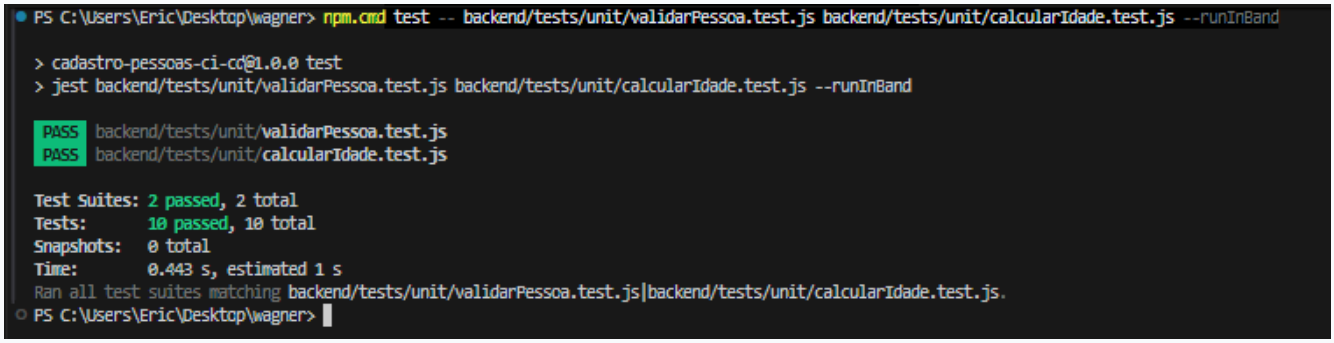
O backend passou a verificar CPF, nome e nascimento, sem consultar serviços externos.



ETAPA 5 - GREEN

Testes passando

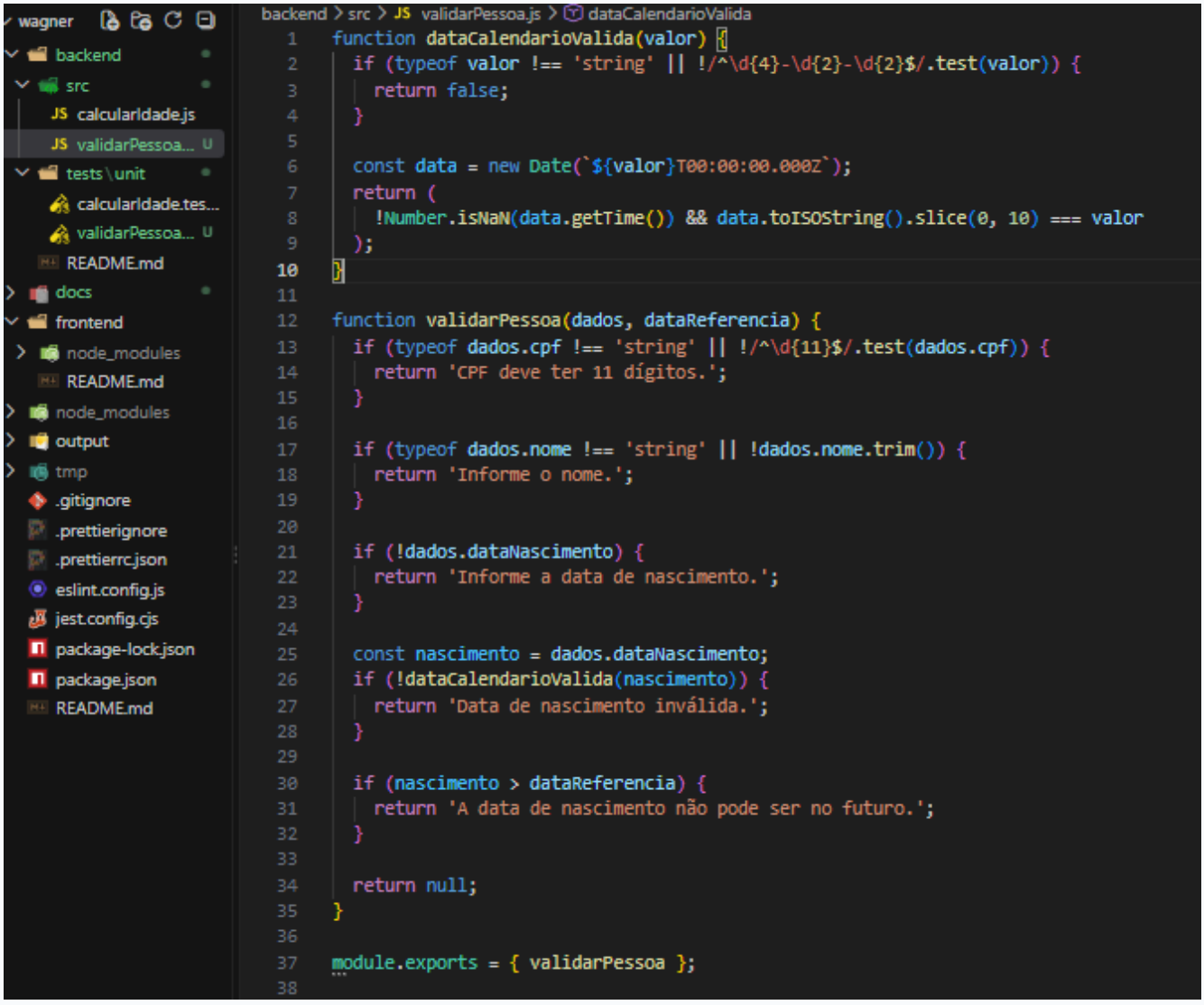
Os sete testes novos e os três testes anteriores passaram após a implementação.



ETAPA 5 - REFACTOR

Código reorganizado

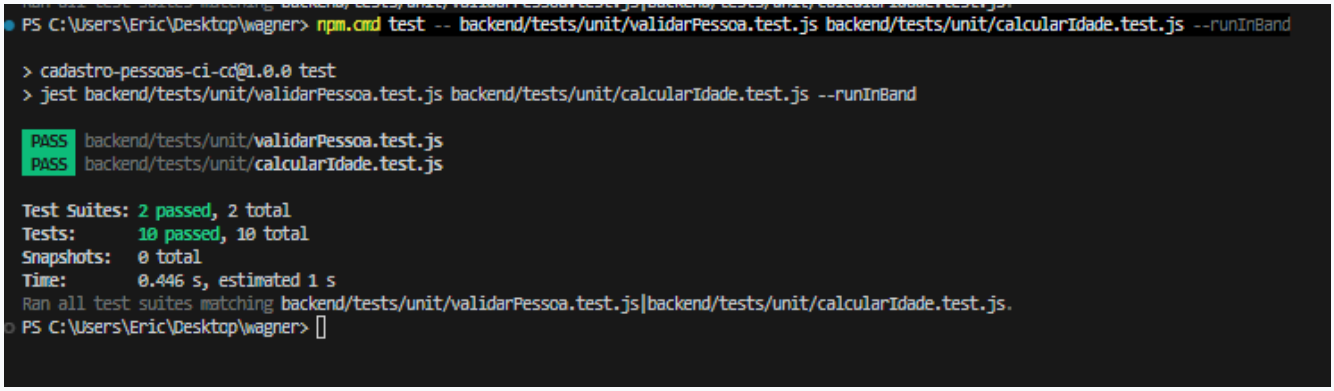
A conferência de data de calendário foi separada, sem mudar as regras testadas.



ETAPA 5 - REFACTOR

Resultados preservados

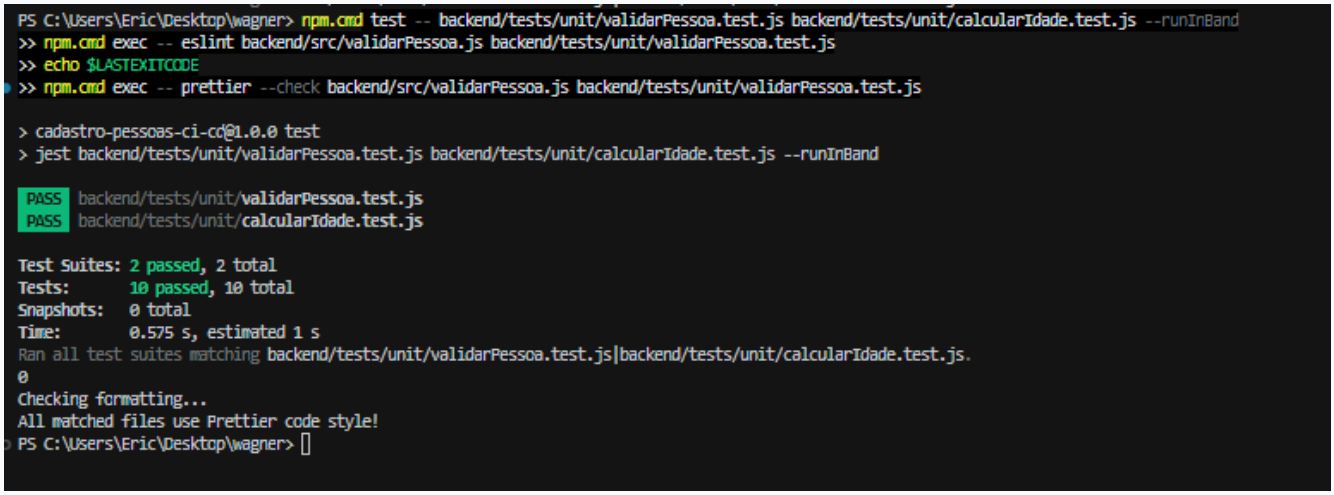
Depois da reorganização, os mesmos dez testes continuaram passando.



ETAPA 5 - VERIFICAÇÕES

Testes e qualidade dos arquivos novos

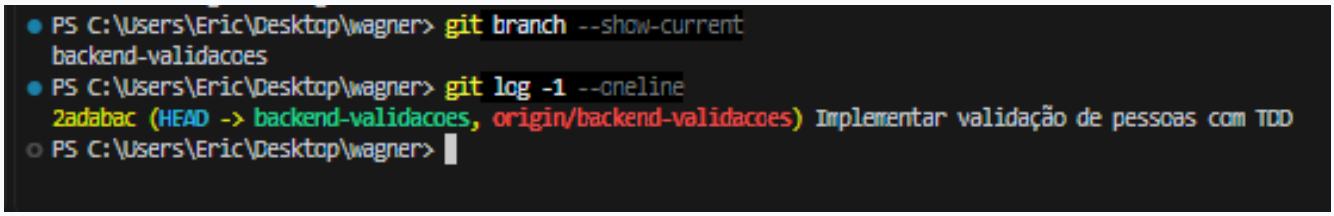
Dez testes passaram; ESLint e Prettier dos arquivos deste ciclo foram aprovados. A formatação geral continua pendente.



ETAPA 5 - GIT

Branch e commit de código

A branch backend-validacoes recebeu o commit 2adabac com o módulo e os testes.



ETAPA 5 - GIT

Pull request para revisão

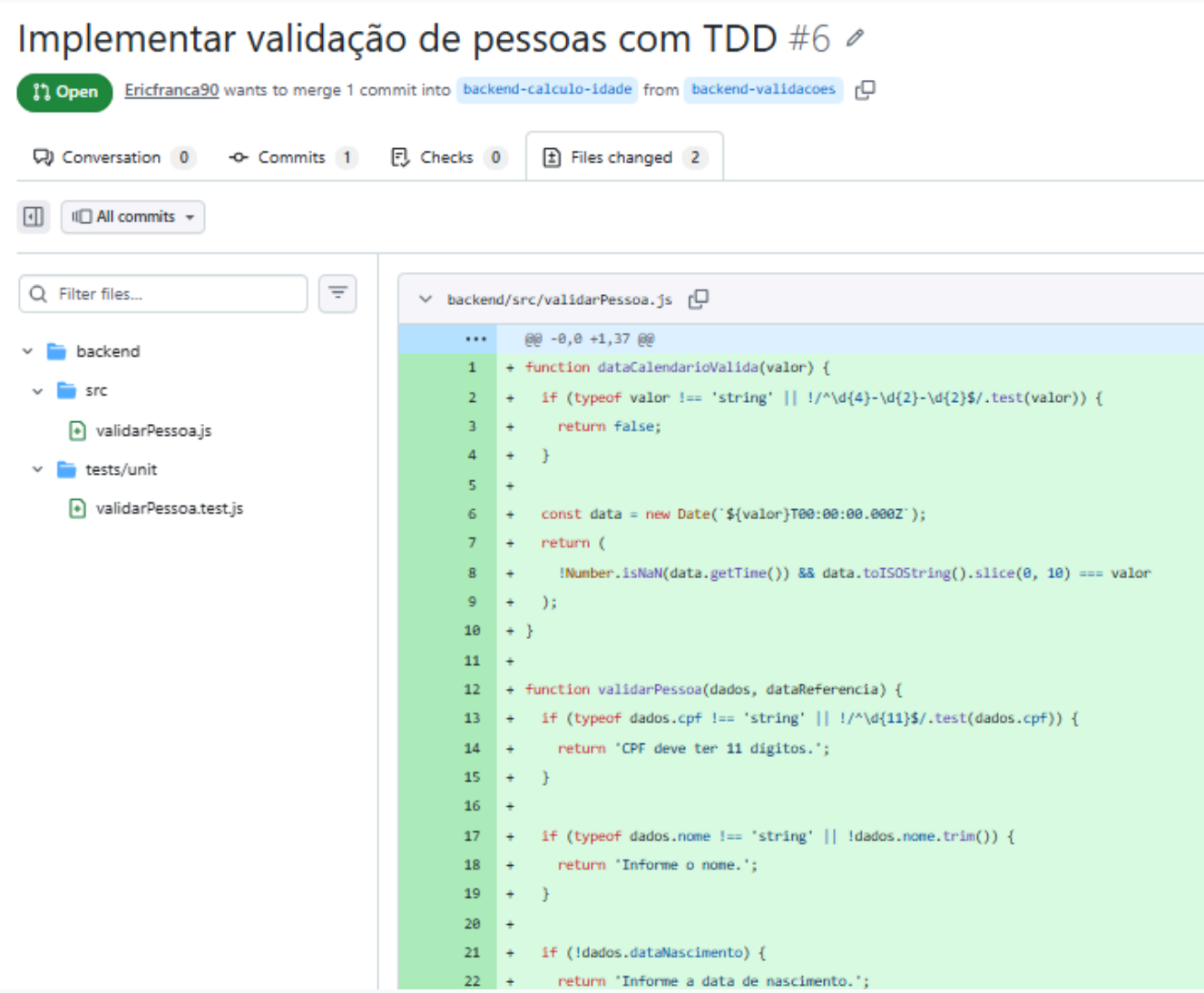
O PR #6 está aberto, de backend-validacoes para backend-calculo-idade, e depende do PR #5.



ETAPA 5 - GIT

Comparação do pull request

A comparação publicada mostra somente o módulo de validação e seus testes, sem reapresentar o cálculo da idade.



INTEGRAÇÃO E ENTREGA CONTÍNUA

Etapa 6
CRUD e API

Projeto Cadastro de Pessoas em JavaScript

> Continuação do relatório acumulado: desenvolvimento do backend com TDD, CRUD, armazenamento JSON e testes. As páginas anteriores preservam as etapas 1 a 5.

Resultado deste bloco: 43 testes passaram; ESLint e Prettier dos arquivos envolvidos passaram.
Versionamento: commit 0b7bb30 e PR #7 aberto sobre backend-validacoes, dependente do PR #6.
Pendência técnica: a verificação geral de formatação aponta cinco arquivos anteriores com finais de linha locais diferentes.
Próximas etapas de Eric: integrar o frontend já desenvolvido, configurar Docker e GitHub Actions.
Data desta atualização: 22 de outubro de 2026.

ETAPA 6 - RED

Teste do primeiro cadastro

O teste foi escrito antes da rota POST, com arquivo temporário e data de referência fixa.



```
describe('POST /pessoas', () => {
  let diretorioTeste;
  let arquivoDados;

  beforeEach(() => {
    diretorioTeste = fs.mkdtempSync(
      path.join(os.tmpdir(), 'cadastro-pessoas-teste-'),
    );
    arquivoDados = path.join(diretorioTeste, 'pessoas.json');
  });

  afterEach(() => {
    if (fs.existsSync(arquivoDados)) {
      fs.unlinkSync(arquivoDados);
    }
    fs.rmdirSync(diretorioTeste);
  });

  test('cadastra a pessoa, calcula a idade e persiste os dados sem a idade', async () => {
    const app = criarApp({
      arquivoDados,
      obterDataReferencia: () => '2026-10-20',
    });

    const resposta = await request(app).post('/pessoas').send({
      cpf: '01234567890',
      nome: '  Ana Souza  ',
      dataNascimento: '2000-05-20',
    });

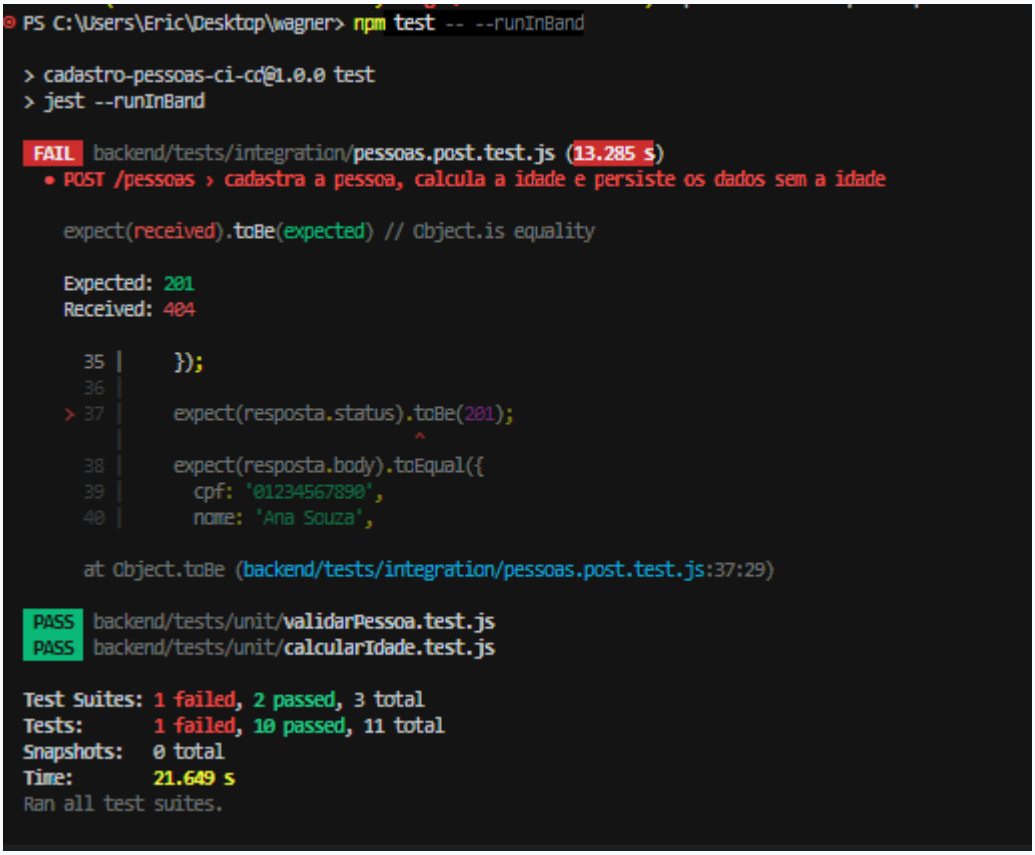
    expect(resposta.status).toBe(201);
    expect(resposta.body).toEqual({
      cpf: '01234567890',
      nome: 'Ana Souza',
      dataNascimento: '2000-05-20',
      idade: 26,
    });

    const cadastros = JSON.parse(fs.readFileSync(arquivoDados, 'utf8'));
    expect(cadastros).toEqual([
      {
        cpf: '01234567890',
        nome: 'Ana Souza',
        dataNascimento: '2000-05-20',
      },
    ]);
  });
```

ETAPA 6 - RED

Falha de comportamento

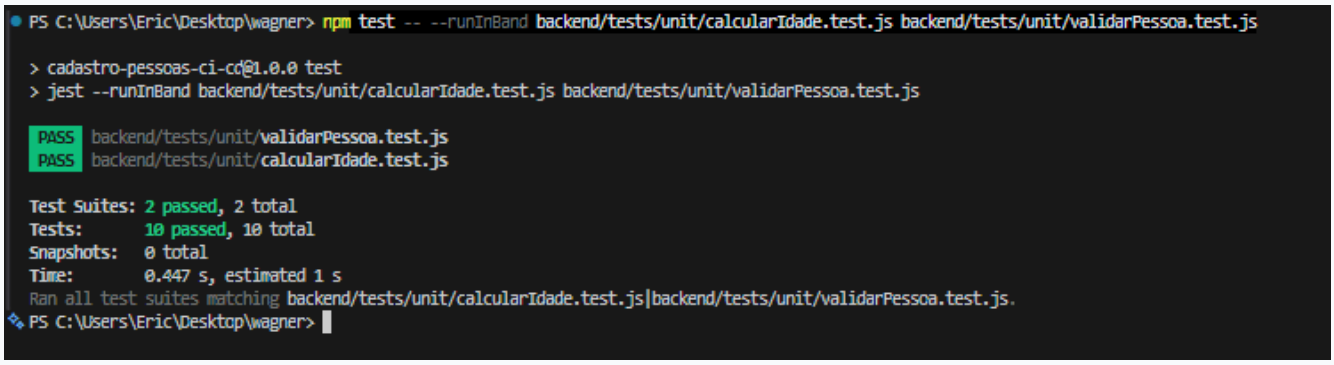
O POST recebeu 404 quando o teste esperava 201; os dez testes anteriores continuaram passando.



ETAPA 6 - RED

Testes anteriores preservados

A execução isolada confirmou que os dez testes de idade e validação continuavam passando.



ETAPA 6 - GREEN

Cadastro implementado

A rota POST valida, normaliza o nome, grava o cadastro e retorna a idade calculada.

app.js U

backend > src > app.js > ...

```
const express = require('express');
const { calcularIdade } = require('./calcularIdade');
const { validarPessoa } = require('./validarPessoa');
const { lerPessoas, salvarPessoas } = require('./armazenamentoPessoas');

function criarApp({ arquivoDados, obterDataReferencia }) {
  const app = express();

  app.locals.arquivoDados = arquivoDados;
  app.locals.obterDataReferencia = obterDataReferencia;
  app.use(express.json());

  app.post('/pessoas', (requisicao, resposta) => {
    const entrada = requisicao.body ?? {};
    const pessoa = {
      cpf: entrada.cpf,
      nome:
        typeof entrada.nome === 'string' ? entrada.nome.trim() : entrada.nome,
      dataNascimento: entrada.dataNascimento,
    };
    const dataReferencia = obterDataReferencia();
    const erro = validarPessoa(pessoa, dataReferencia);

    if (erro) {
      return resposta.status(400).json({ erro });
    }

    const pessoas = lerPessoas(arquivoDados);
    pessoas.push(pessoa);
    salvarPessoas(arquivoDados, pessoas);

    return resposta.status(201).json({
      ...pessoa,
      idade: calcularIdade(pessoa.dataNascimento, dataReferencia),
    });
  });

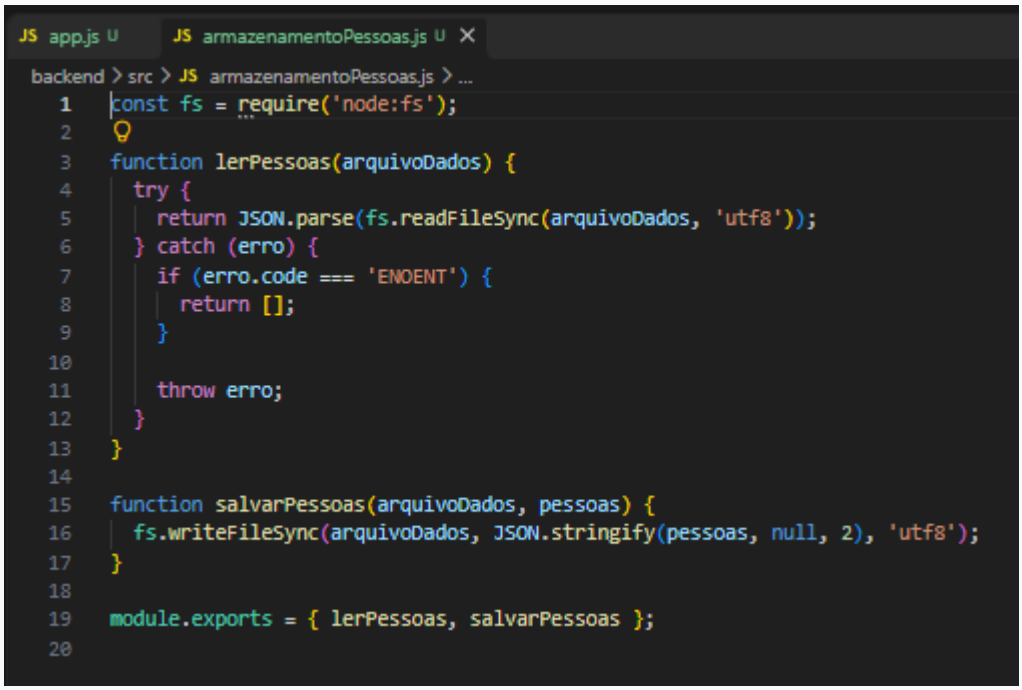
  return app;
}

module.exports = { criarApp };
```

ETAPA 6 - GREEN

Armazenamento JSON

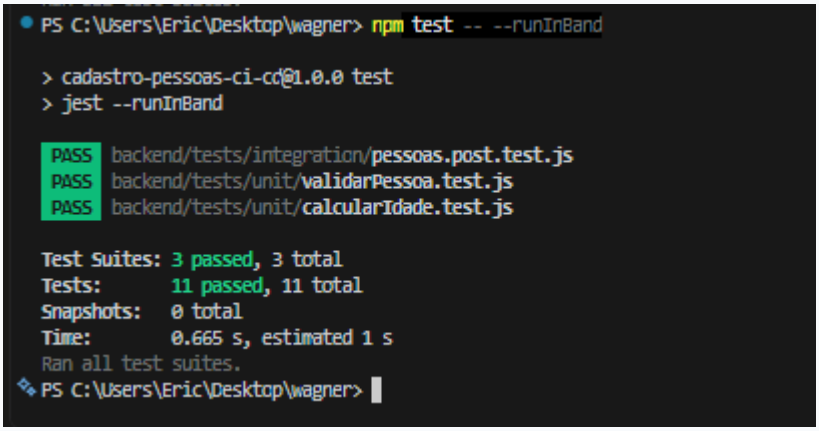
A leitura e a gravação usam arquivo JSON local; a idade não é persistida.



ETAPA 6 - GREEN

Primeiro ciclo aprovado

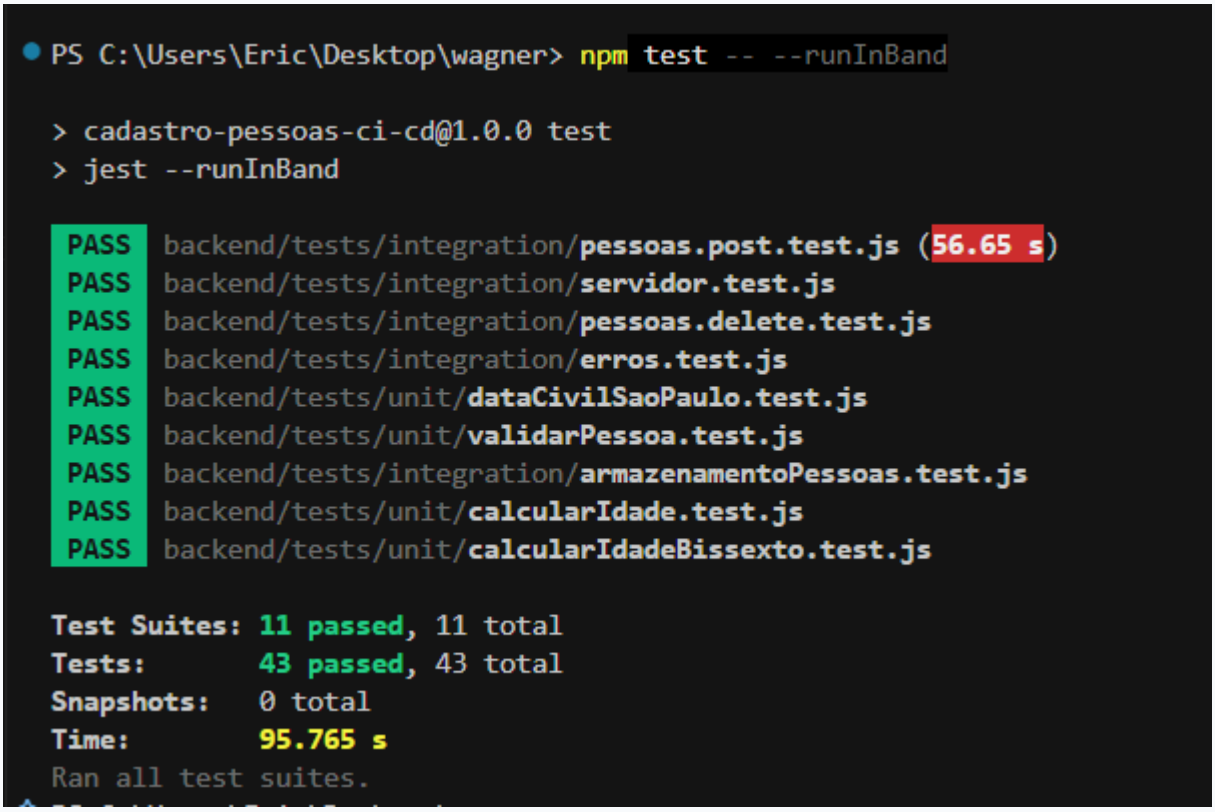
Depois da implementação do POST, onze testes passaram: o novo teste e os dez anteriores.



ETAPA 6 - TESTES

Suíte completa do backend

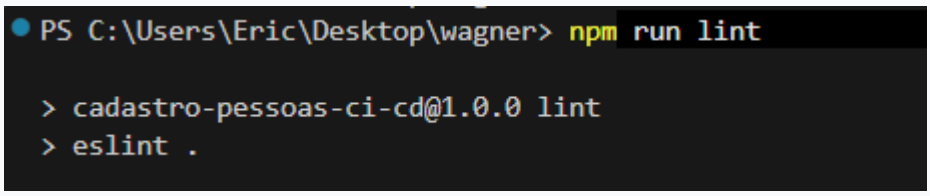
Após os ciclos de listagem, edição, exclusão, armazenamento e erros, 43 testes passaram.



ETAPA 6 - QUALIDADE

Análise estática

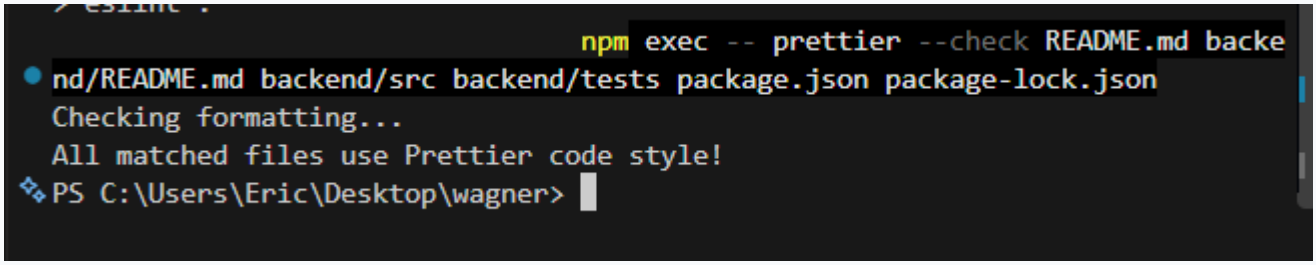
O ESLint foi executado para o projeto; a verificação concluída não apontou erros.



ETAPA 6 - QUALIDADE

Formatação dos arquivos deste bloco

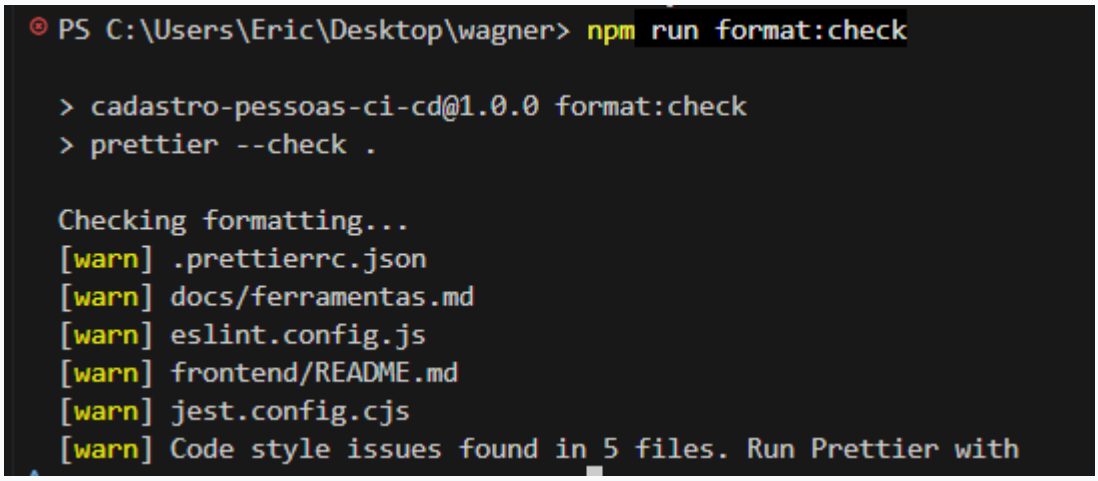
O Prettier confirmou o padrão de código nos arquivos alterados e criados para o backend.



ETAPA 6 - PENDÊNCIA

Formatação geral

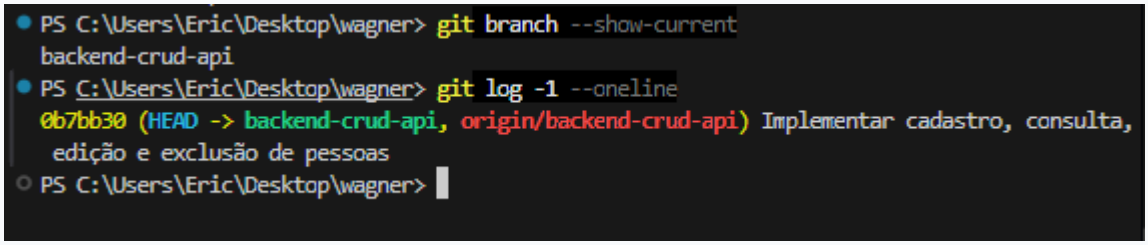
A verificação geral ainda aponta cinco arquivos anteriores; esse resultado não foi apresentado como aprovação.



ETAPA 6 - GIT

Branch e commit

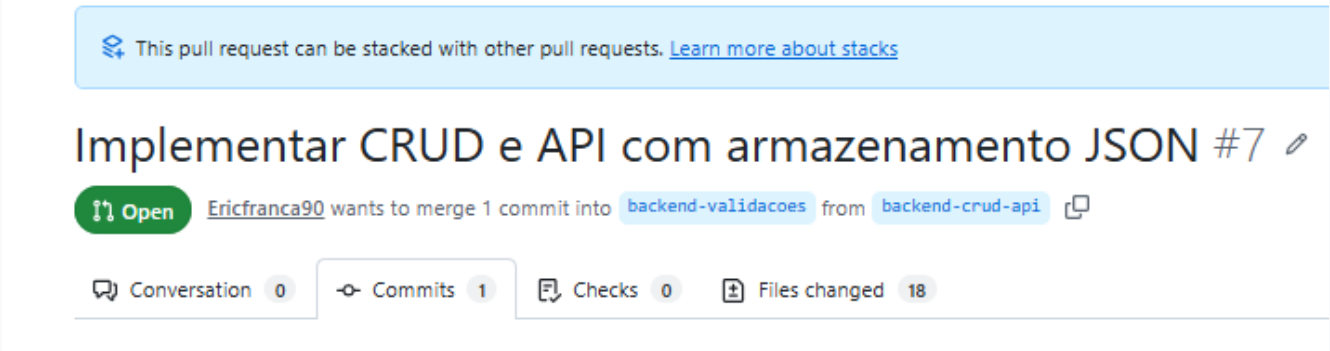
A branch backend-crud-api registra o commit 0b7bb30 com o código e os testes.



ETAPA 6 - GIT

Pull request aberto

O PR #7 foi aberto de backend-crud-api para backend-validacoes e depende do PR #6.



ETAPA 6 - GIT

Comparação do pull request

A aba Files changed mostra os 18 arquivos deste bloco, sem alterações no frontend.

